



RH134

Managing Basic Storage



Red Hat | Enterprise Linux

Goal

- Create and manage storage devices, partitions, file systems, and swap spaces from the command line.

Objectives

- Create storage partitions, format them with file systems, and mount them for use.
- Create and manage swap spaces to supplement physical memory.

File Systems

تُعد أنظمة الملفات (File Systems) مسؤولة عن إدارة وتنظيم البيانات المخزنة على وسائط التخزين المختلفة، وتحديد كيفية حفظها واسترجاعها بفاعلية.

1. أنواع وسائط التخزين (Storage Devices)

- HDD & SSD: وسائط التخزين التقليدية وأقراص الحالة الصلبة.
- NVMe: نوع أحدث بكثير من الـ SSD، ويتميز بسرعات فائقة وأداء عالٍ جداً (High Performance).
- وسائط أخرى: مثل Memory Card، و USB Flash Drive، وملفات الأقراص مثل CD/DVD ISO.

2. أنواع أنظمة الملفات (File Systems)

يدعم نظام لينكس مجموعة متنوعة من أنظمة الملفات، والتي تنقسم حسب استخداماتها وبيئتها:

○ أنظمة ملفات خاصة بأنظمة لينكس (Linux Native File Systems):

- Ext3 & Ext4: تُعد من أكثر أنواع أنظمة الملفات شيوعاً واستخداماً في السوق (Most Common Use for Market)، وتتميز بالاستقرار والأداء العالي.

- XFS File System: نظام ملفات قوي وعالي الكفاءة، وغالباً ما يُستخدم افتراضياً في توزيعات لينكس للتعامل بكفاءة مع المساحات الهائلة والملفات الضخمة.

○ أنظمة ملفات أخرى متوافقة (Cross-Platform File Systems):

- NTFS: نظام الملفات الأساسي في أنظمة ويندوز، ويدعم نظام لينكس التعامل معه.
- FAT & exFAT: أنظمة ملفات شائعة الاستخدام مع وسائط التخزين الخارجية (مثل USB Flash Drive و Memory Card) لضمان التوافقية وسهولة نقل البيانات بين مختلف أنظمة التشغيل (ويندوز، ماك، ولينكس).

What is a file system in Linux or Unix?

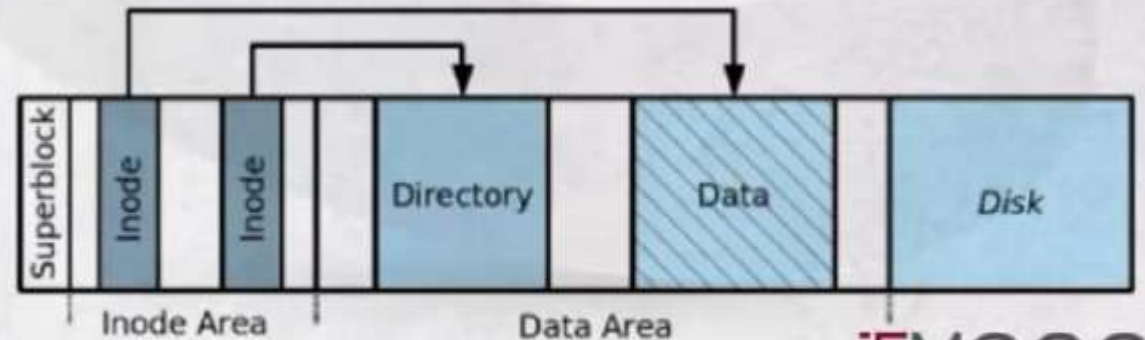
- File system is a collection of files and directories stored on disk. Each file system is stored in a separate whole disk partition. The following are a few of the file system:

But what is in a File system?

- File system divided into two categories:
 - **User data** – Stores actual data contained in files.
 - **Metadata** – Stores file system structural information such as superblock, inodes, directories.

Partition divided into blocks

- Super block
- Inode blocks
- Data block



Each file system is stored in a separate whole disk partition :

التوضيح:

تعني أن كل نظام ملفات (File System) مستقل بذاته ويتم إنشاؤه وتخزينه داخل بارتيشن (Partition) أو تقسيم منفصل بالكامل على القرص الصلب، بحيث لا يتداخل نظاما ملفات مختلفان داخل نفس البارتيشن، بل يكون لكل واحد مساحة مخصصة بمفرده.

شرح مفهوم الـ File System:

- **User Data**
 - يخزن البيانات الفعلية التي تحتويها الملفات مثل الـ **images**، والـ **videos**، والـ **text**، والـ **application data**.
- **Metadata**
 - يخزن المعلومات الهيكلية للـ **file system** مثل الـ **superblock**، والـ **inodes**، والـ **directories**.
- **Super block**
 - يتواجد عادة في بداية الـ **file system** ويخزن معلومات شاملة عن الـ **file system** مثل الـ **file system type** المستخدم وبعض المعلومات الأخرى.
- **Inode blocks (Inode Area)**
 - عبارة عن قاعدة بيانات أو **tables** فيها مجموعة من الـ **rows**، كل **row** فيهم فيه معلومات عن فايل أو **directory** معين، زي الـ **file type**، والـ **permissions**، والـ **owner**، والـ **Access Control Lists (ACL)** وهكذا.
 - كل فايل أو **inode** سيكون له **inode number**، أما الـ **file name** فلا يخزن في الـ **inode** ولكنها تخزن في الـ **directory**، ويكون عبارة عن رابط بين الـ **inode number** والـ **file system** للـ **file**.
- **Data block (Data Area)**
 - المنطقة المخصصة داخل الـ **partition** لتخزين البيانات الفعلية للملفات والمجلدات.

What is Superblock?

- Each file system is different, and they have type like ext2, ext3 etc.
- Further each file system has size like 5 GB, 10 GB and status such as mount status. In short, each file system has a superblock, which contains information about file system such as:

- File system type
- Size
- Status
- Information about other metadata structures

○ حالة ال Mount Status:

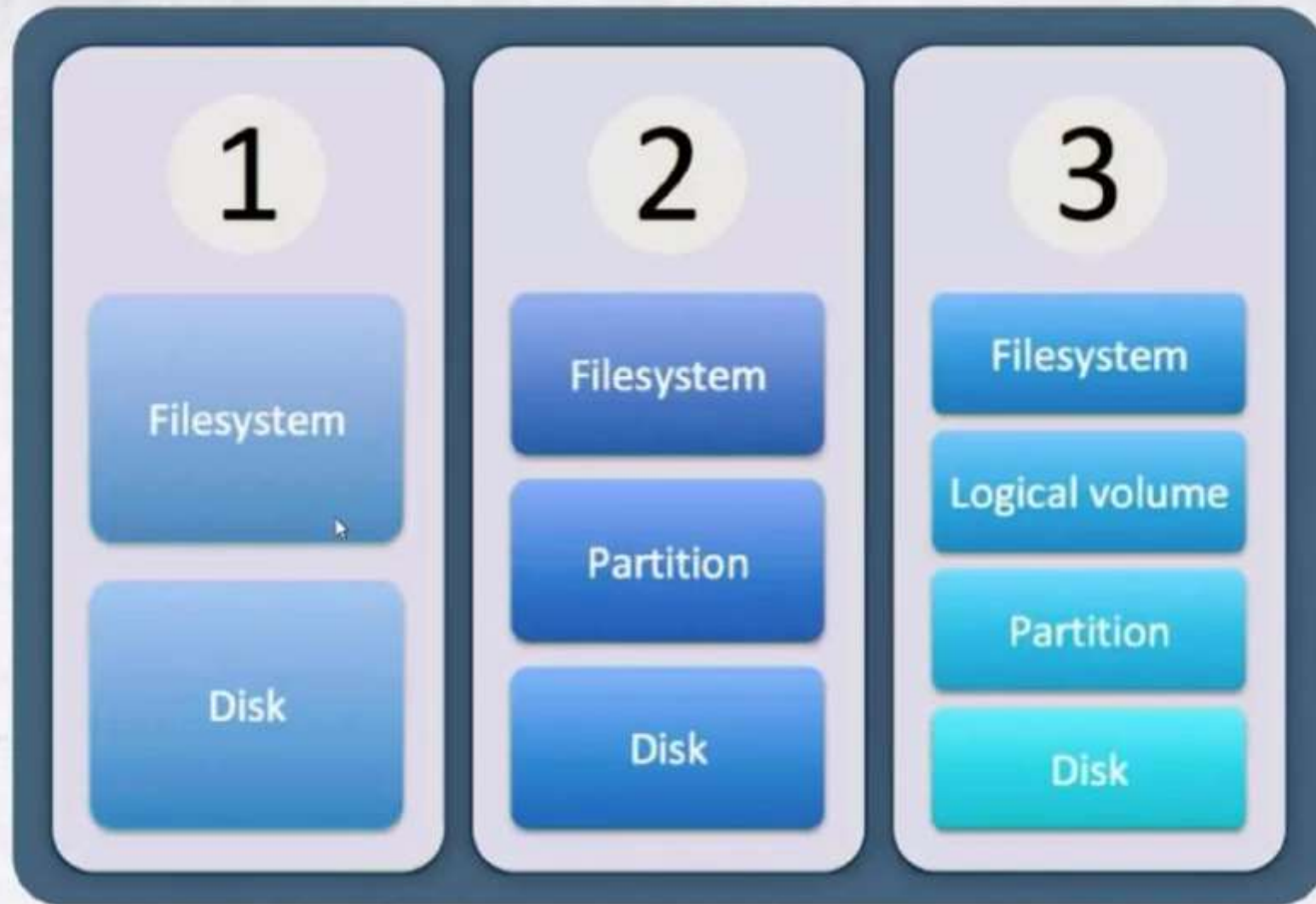
- تعني ببساطة ما إذا كان ال **file system** "مربوطاً" ومتاحاً للاستخدام الفعلي من قبل نظام التشغيل والمستخدمين، أو غير مربوط.
- بمعنى أقرب: زي ما توصل فلاش ميموري باللاب توب؛ لو ظهرت وتقدر تدخل تفتحها وتقرأ وتكتب عليها، يبقى كدة حصل لها **Mount** (متاحة للاستخدام)، أما لو لسه متوصلتش بالشكل اللي يخلي النظام يتعامل مع ملفاتها، بتبقى **Unmounted** (غير متاحة).

What is Inodes?

➤ Metadata about the files

- File type (executable, block special etc)
- Permissions (read, write etc)
- Owner
- Group
- File size
- File access, change and modification time
- Number of links (soft/hard)
- Access Control List (ACLs)

Adding Disks Partitions and File Systems to a Linux System



طرق إضافة واستخدام الأقراص (Adding Disks and File Systems):

○ الطريقة الأولى (استخدام القرص كاملاً بدون تقسيم):

- يتم عمل فورمات للـ **disk** بالكامل مباشرة بـ **file system** معين (مثل XFS) بناءً على طلب الـ **vendor** أو الحاجة الإدارية.
- لا يكفي عمل الفورمات فقط، بل يجب عمل **mount** للقرص على **mount point** لكي يصبح متاحاً للاستخدام.
- لضمان بقاء الـ **mount** واستمراره حتى بعد إعادة تشغيل السيرفر (Reboot)، يتم إضافته وإجهاذه في ملف الـ `etc/fstab/`.

○ الطريقة الثانية (تقسيم القرص التقليدي - Partitions):

- يتم تقسيم الـ **disk** إلى أكثر من **partition**، وكل **partition** يمكن عمل فورمات له بـ **file system** مختلف حسب الرغبة.
- بعد تقسيم الـ **disk**، يتم إنشاء الـ **file system** على كل بارتيشن، ثم عمل **mount** مباشر له.

○ الطريقة الثالثة (التخزين المنطقي - Logical Volume / LVM):

- **نقطة سريعة:** هي طريقة متقدمة ومرنة جداً لإدارة التخزين (سيتم تفصيلها في الشاتر القادم). تعتمد على وضع طبقة وسيطة بين الـ **partition** والـ **file system** تُسمى الـ **logical volume**.
- **أهميتها:** تتيح لك إمكانية توسيع أو تقليص مساحة الـ **file system** والبارتيشنز بسهولة شديدة ودون فقدان البيانات ودون التقيود بالحدود الفيزيائية الصلبة للقرص.

Partitioning a Disk

- Disk partitioning allows system administrators to divide a hard drive into multiple logical storage units, referred to as partitions. By separating a disk into partitions, system administrators can use different partitions to perform different functions.
- **For example, disk partitioning is necessary or beneficial in these situations:**
 - Limit available space to applications or users.
 - Separate operating system and program files from user files.
 - Create a separate area for memory swapping.
 - Limit disk space use to improve the performance of diagnostic tools and backup imaging.

فوائد وأسباب تقسيم الأقراص (Partitioning a Disk):

○ **Limit available space to applications or users:**

○ **الشرح:** تحديد مساحة معينة لا يمكن تجاوزها. زي ما قلت تماماً، لما تعمل **partition** لمجلد **home/** وتديله 10 جيجابايت فقط، فأنت كدة بتعمل **limit** لمساحة الـ **regular users** بحيث لا يمكنهم استهلاك أكثر من هذه المساحة، وده بيمنع أي مستخدم أنه يملأ الهارد كله لوحده.

○ **Separate operating system and program files from user files:**

○ **الشرح:** فصل ملفات نظام التشغيل والبرامج عن ملفات المستخدمين. دي ميزة **isolation** (عزل)، بمعنى لو حصلت مشكلة أو عطل في النظام أو البرامج واضطرتت تعمل فورمات أو إعادة تثبيت لملفات الـ **OS**، فملفات المستخدمين الشخصية هتفضل في **partition** ثاني خالص بعيدة ومعزولة تماماً ومش هيحصل لها أي حاجة أو فقدان.

○ **Create a separate area for memory swapping:**

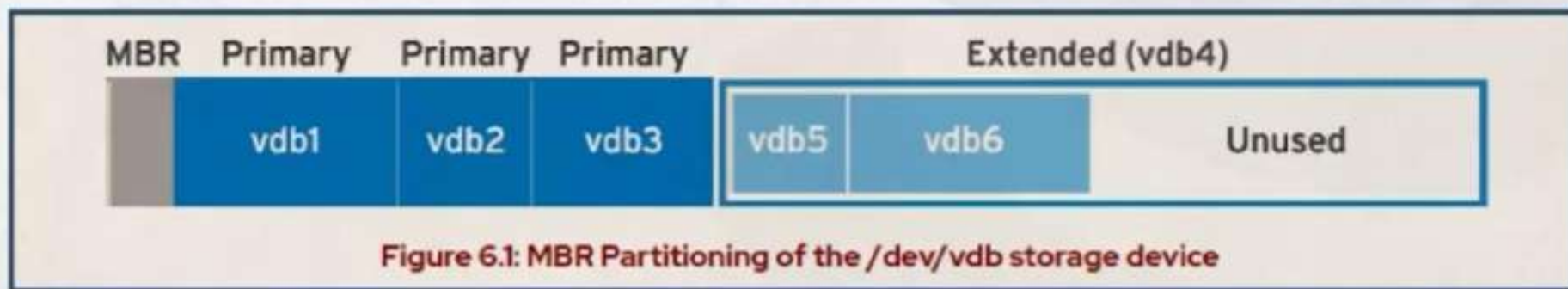
○ **الشرح:** تخصيص مساحة منفصلة للـ **swap space** (الذاكرة التبادلية). لما الرامات (RAM) تتملي بالكامل، نظام التشغيل بياخد جزء من الهارد ده عشان يشتغل كأنه رامات إضافية مؤقتة، وفصلها في **partition** لوحدها بيحسن أداء النظام وإدارة الذاكرة.

○ **Limit disk space use to improve the performance of diagnostic tools and backup imaging:**

○ **الشرح:** تحديد مساحة الاستخدام لتسهيل وتحسين أداء أدوات الصيانة والفحص وأخذ النسخ الاحتياطية (**backup imaging**). لما يكون البارتيشن حجمه صغير ومحدد، عمليات مثل فحص الأخطاء أو عمل **image backup** بتتم بشكل أسرع بكثير وبكفاءة أعلى مقارنة بالتعامل مع ديسك ضخم كله حته واحدة.

MBR Partitioning Scheme

- Since 1982, the Master Boot Record (**MBR**) partitioning scheme has dictated how disks are partitioned on systems. This scheme supports a maximum of **four primary partitions**. On Linux systems, with the use of extended and logical partitions, administrators can create a maximum of **15 partitions**. Because partition size data is stored as **32-bit values**, disks partitioned with the MBR scheme have a maximum disk and partition size of **2 TiB**.



- The **2 TiB disk** and partition size limit of the MBR partitioning scheme is no longer a theoretical limit but rather a real-world problem that system administrators encounter more and more frequently in production environments. As a result, the legacy **MBR scheme** is in the process of being superseded by the new **GUID Partition Table (GPT)** for disk partitioning.

أنواع ال Partition Schemes في اللينكس والسيستم:

ال Partition schema باختصار هي ال structure (الهيكل أو النظام التنظيمي) المتبع لتنظيم وتقسيم ال storage device (وحدة التخزين) إلى مجموعة من ال logical sections أو ال partitions الأصغر لكي يتمكن نظام التشغيل من التعامل معها وتخزين البيانات بداخلها. والنوعين الأساسيين المستخدمين هما MBR و GPT، وسنتناول هنا تفاصيل نظام ال MBR بالكامل مع كل الشرح وال limitations التي ذكرها الإنستراكتور وما ورد في السلايد:

مفهوم ونشأة نظام ال MBR:

منذ عام 1982، نظام ال Master Boot Record (MBR) partitioning scheme هو الذي يحكم كيفية تقسيم الأقراص على الأنظمة.

حدود عدد البارتيشنز في ال MBR:

- النظام ده بيدعم بحد أقصى أربعة بارتيشنز أساسية (Primary partitions).
- حتى لو فاضل مساحة في الديسك وحببت تزود عدد البارتيشنز، الحل إنك تعمل ثلاثة بارتيشنز أساسية (Primary) وواحد Extended.
- ال Extended ده بتعرف تعمل جواه أكثر من بارتيشن (Logical partitions).
- طبقًا آخرك تكريت بارتيشن Extended واحد فقط، فدايمًا خلي ال size بتاعه اللي هو ال Free أو ال Available size للديسك كله، اديه لل Extended كامل.

Limitations وعيوب ال Extended وال MBR:

- **تصحيح النقطة الخاصة بال Extended:** ال Extended partition في حد ذاته لا يُستخدم لتخزين البيانات مباشرة، بل هو حاوية أو غلاف يتم إنشاؤه لتقسيمه من داخله إلى عدة Logical partitions. أي أنك لا تضع عليه file system بشكل مباشر، بل تضع ال file system على البارتيشنز المنطقية (Logical partitions) التي بداخله.
- بعض أنظمة التشغيل (Operating Systems) تتطلب منك إنك تعمل لها Restore أو تثبيت على Primary partition زي الويندوز وبعض توزيعات اللينكس، لكن في ال Red Hat مفيش أي مشاكل في التعامل مع ال Logical partitions.
- بالطريقة ال Extended عندك تو أوبشن فقط: إما تكرت 4 Primary partitions ومش هتعرف تعمل New partitions بعدها لو احتجت، يا إما تكرت 3 Primary وواحد Extended، وده بيسمح لك تعمل بحد أقصى 15 بارتيشن في المجموع (3 Logical + 12 Primary).
- البيانات في ال MBR بتبقى stored as 32-bit values، وده بيخلي ال maximum partition value أو ال maximum disk size يمكن إنشاؤه على ال MBR disk هو 2 تيرابايت (2 TiB)، ومش هتعرف تعمل بارتيشن أكبر من كده نهائياً.

هيكل ال MBR (ال MBR Sector Structure):

هتلاحظ في ال MBR disk إن في سيكشن اسمه MBR، السيكشن ده بيبقى موجود في أول الديسك خالص، ال size بتاعه بيبكون 512 بايت ومقسوم إلى ثلاثة أجزاء رئيسية:

- الجزء الأول (446 بايت): بيبكون جوه حاجة اسمها ال Boot strap code، وده اللي بيليود البوت لودر (Boot loader) اللي بيقوم السيستم وال boot up بتاعك.
- الجزء الثاني (64 بايت): بيبكون اسمه Partition table، بيبكون جوا معلومات عن البارتيشنز اللي في الديسك بتاعك زي ال start sector وال end sector وحجم البارتيشن وهل هو active أو bootable (يعني هل هو البارتيشن النشط الذي يحتوي على ملفات الإقلاع أم لا).
- الجزء الثالث (2 بايت): بيبكون جوه حاجة اسمها Boot signature (أو ال magic number)، وبيكون عبارة عن علامة البيوس (BIOS) بيستخدمها عشان يتأكد هل ال MBR sector ده valid وصالح ولا corrupted.

عيوب إضافية ومشاكل ال MBR في البيئات الحديثة:

- مفيش أي Redundancy في ال MBR (أي لا يوجد نسخة احتياطية لجدول التقسيم)، يعني لو ال data بتاعتك حصل لها corruption، احتمال كبير ت lose the data بتاعتك كلها وما تعرّفش ترجعها تاني.
- الحد الأقصى TiB 2 للديسك والبارتيشن مابقاش لميت نظري بس، ده بقى مشكلة حقيقية وواقعية بيقابلها ال system administrators بشكل متكرر في ال production environments مع المساحات الضخمة للديسكات حالياً.
- نتيجة لكل العيوب والحدود دي، ال legacy MBR scheme في طريقها لأن يتم استبدالها بالكامل بنظام ال GUID Partition Table (GPT) الحديث للتقسيم.

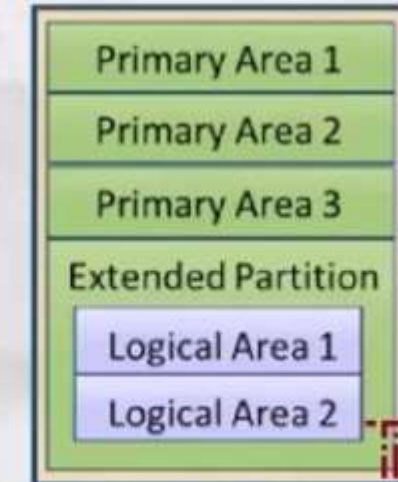
Primary, Extended, and Logical partition types

- **Primary partition** – These are the convention partitions that would create by default. If you only plan to create 4 partitions or less, then you only need to create primary partition and you don't have to worry about extended and logical partitions.
- **Extended partition** – If you want to create more than 4 partitions on a hdd, now or in the future, then you need to create one of the partition as an extended partition. An extended partition is not actually a partition, instead it is essentially an empty container inside which you can create (logical) partitions. You cannot store data inside an extended partition directly, instead you first have to create logical partition inside the extended partition. You can only create one extended partition per hdd.
- **Logical partition** – These are the names of any partitions created inside an extended partition, OS uses logical partitions in the same was as a typical primary partition.

Primary Area Only

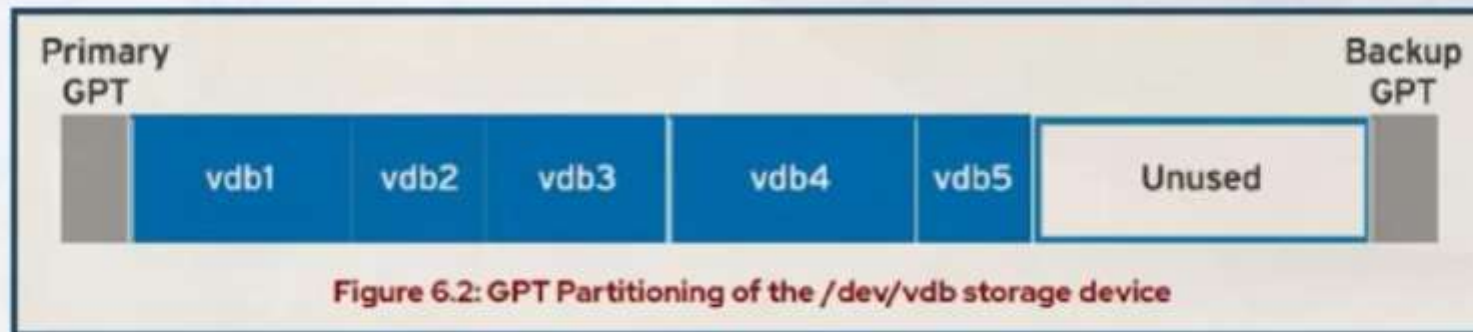


Extended Area Only



GPT Partitioning Scheme

- A GPT provides a maximum of **128 partitions**. Unlike an MBR, which uses 32 bits for storing logical block addresses and size information, a GPT allocates **64 bits** for logical block addresses.
- Allows a GPT to accommodate partitions and disks of up to eight zebibytes (ZiB) or eight billion tebibytes.
- In addition to addressing the limitations of the MBR partitioning scheme, a GPT also offers some additional features and benefits. A GPT uses a **globally unique identifier (GUID)** to identify each disk and partition. In contrast to an MBR, which has a single point of failure, a GPT offers redundancy of its partition table information. The primary GPT resides at the head of the disk, while a backup copy, the secondary GPT, is housed at the end of the disk. A GPT uses a checksum to detect errors and corruptions in the GPT header and partition table.



شرح نظام ال GPT Partitioning Scheme ومقارنته بال MBR:

- عدد البارتيشنز (Maximum Partitions):
 - نظام ال GPT يوفر بحد أقصى 128 بارتيشن على الديسك الواحد، مقارنة بال MBR الذي كان محدوداً بـ 4 بارتيشنز رئيسية.
- حجم المساحة والبيانات المستخدمة (Block Addresses & Size):
 - على عكس ال MBR الذي كان يستخدم bit-32، فإن ال GPT يخصص bit 64 لعناوين ال logical blocks.
 - هذا يتيح لنظام ال GPT استيعاب بارتيشنز وأقراص صلبة تصل مساحتها إلى 8 زيبيبايت (ZiB) أو ما يعادل حوالي 8 مليار تيبيبايت (رقم خيالي يحل نهائياً مشكلة ال 2 تيرا الخاصة بال MBR).
- ميزات إضافية ومعرف فريد (GUID):
 - يستخدم ال GPT معرفاً فريداً عالمياً يُعرف بـ (globally unique identifier GUID) لتمييز كل ديسك وكل بارتيشن بدقة.
- ال Redundancy وتأمين البيانات (Primary vs. Backup GPT):
 - على عكس ال MBR الذي كان يمتلك نقطة فشل واحدة، يوفر ال GPT خاصية ال redundancy لمعلومات جدول التقسيم (partition table).
 - ال Primary GPT يتم وضعه في بداية الديسك، بينما توجد نسخة احتياطية مطابقة وهي ال secondary GPT (أو ال Backup GPT) يتم تخزينها في نهاية الديسك.
- يستخدم ال GPT نظام تحقق (checksum) لكشف أي أخطاء أو تلف في ال GPT header وجدول التقسيم.

ملحوظة هامة حول حقيقة عمل ال Backup GPT:

○ استنتاجك صحيح تماماً؛ ال Backup GPT الموجود في نهاية الديسك لا يحفظ نسخة من الداتا الفعلية أو الملفات التي تقوم بتخزينها داخل البارتيشنات.

○ كل ما يقوم بحفظه وعمل نسخة احتياطية منه هو "خريطة" جدول التقسيم (Partition Table structure) فقط (أي أماكن بداية ونهاية كل بارتيشن، ومساحاتهم، وأسمائهم، وال GUID الخاص بهم).

○ بالتالي، لو حصل تلف (corruption) بسبب هجوم برمجي (attack) أو أي سبب آخر في ال Primary GPT، فالسيستم أو الهارد لن يصبح قادراً على معرفة بداية أي بارتيشن من أي حته، ولا عناوين أي حاجة خالص، ولن يعرف هل ال data اللي جوه كل بارتيشن آمنة ولا لأ أو في مكانها ولا لأ، ونتيجة لذلك يمكن أن يظهر لك الهارد كأنه مساحة غير تخصيصية (unallocated).

○ في هذه اللحظة، يتدخل النظام ليقوم بالذهاب لل backup GPT الموجود في نهاية الديسك، فيجلب منه خريطة جدول التقسيم السليمة ويستعيدها، فترجع الدنيا لطبيعتها وتظهر البارتيشنات وبياناتك بداخلها بأمان دون أن تتأثر.

Should I Use MBR or GPT in Linux?

One of the biggest advantages of **GPT** over **MBR** is, on an MBR disk, the partitioning and boot data is stored in one place. If this data is corrupted, you're in trouble meanwhile in GPT stores multiple copies of this data across the disk, so you can recover if the data is corrupted.

مقارنة واختيار الأفضل بين MBR و GPT في اللينكس (Should I Use MBR or GPT in Linux):

العوامل المحددة للاختيار (Factors):

- اختيار النوع الأفضل يعتمد على مجموعة من العوامل الأساسية التي يحددها لك النظام واحتياجاتك.

- العامل الأول (توافقية النظام - Compatibility):** هل السيستم أو اللوحة الأم (Motherboard) تدعم النوعين أم أن الأجهزة القديمة تدعم MBR فقط؟ (الأنظمة الحديثة تدعم GPT بصورة أساسية عبر الـ UEFI، بينما الأجهزة القديمة جداً تعتمد على BIOS تقليدي مع MBR).

- العامل الثاني (حجم المساحة - Storage Size):** هل حجم الديسك فارق معك أم لا؟ إذا كان لديك مساحة صغيرة ولن تتعدى 2 تيرابايت، فيمكنك استخدام MBR أو GPT براحتك عادي.

- العامل الثالث (المساحات الضخمة وعدد البارتيشنز):** في حال كان لديك ديسك مساحته كبيرة جداً أكبر من 2 تيرابايت وتحتاج إلى مجموعة كبيرة جداً من البارتيشنز (تصل إلى 128 بارتيشن)، فهنا يجب عليك حتماً استخدام GPT.

الميزة الكبرى لـ GPT في حماية البيانات (محتوى السلايد):

- من أكبر مميزات الـ GPT مقارنة بالـ MBR هي طريقة تخزين بيانات التقسيم والإقلاع.

- في هارد الـ MBR، يتم تخزين بيانات التقسيم والبوت في مكان واحد فقط (في أول الديسك)، وإذا تلف أو حدث له corruption، فأنت في مشكلة كبيرة.

- أما في الـ GPT، فهو يقوم بتخزين نسخ متعددة من هذه البيانات عبر الديسك (نسخة أساسية في الأول ونسخة احتياطية في النهاية)، بحيث يمكنك استعادة جدول التقسيم إذا حدث أي تلف أو corruption في البيانات.

Managing Partitions With **Parted**

- Partition editors are programs which allow administrators to make changes to a disk's partitions, such as creating partitions, deleting partitions, and changing partition types. To perform these operations, administrators can use the Parted partition editor for both the MBR and the GPT partitioning scheme.
- The **parted** command takes the device name of the whole disk as the first argument and one or more subcommands. The following example uses the **print** subcommand to display the partition table on the **/dev/vda** disk.

```
[root@host ~]# parted /dev/nvme0n1 print
```

- If you do not provide a subcommand, **parted** opens an interactive session for issuing commands.

```
[root@host ~]# parted /dev/nvme0n1
```

إدارة البارتيشن باستخدام أداة (Parted (Managing Partitions With Parted:

○ مفهوم أداة Parted:

- أدوات تعديل البارتيشن (Partition editors) هي برامج تتيح لمديري النظام (administrators) إجراء تعديلات على أقسام الهارد ديسك، مثل إنشاء البارتيشن، وحذفها، أو تغيير أنواعها.
- يمكنك استخدام أداة Parted للتعامل مع كلا نوعي التقسيم: MBR و GPT بكفاءة وسهولة.

○ قاعدة هامة جداً قبل البدء:

- أي تعديل أو تغيير (change) تقوم بتنفيذه باستخدام أمر parted يتم تنفيذه وتطبيقه بشكل فوري (Takes effect immediately).
- لذلك يجب أن تكون حذراً للغاية (Be careful) وأنت شغال به، لأن أي خطأ أو غلطة (mistake) ممكن تؤدي إلى ضياع الداتا (loss data) بشكل نهائي.

○ طريقتين (Options) لاستخدام ستاكس أو أمر parted:

- الطريقة الأولى (Command-line with subcommands): أن تكتب أمر parted وتديله اسم الديسك كأول معامل، وبعده تكتب ال subcommand المباشر الذي تريد تنفيذه دفعة واحدة.
- مثال: parted /dev/nvme0n1 print (هنا أمر print يعرض لك البارتيشن تيبه والبارتيشن الموجودة داخل الديسك دهوت مباشرة).
- الطريقة الثانية (Interactive Session): إذا لم تقم بكتابة subcommand مع الأمر، فإن parted سيفتح لك جلسة تفاعلية (interactive session) خاصة بالبارتيشن كوماند.
- مثال: parted /dev/nvme0n1 (من خلال هذه الجلسة التفاعلية المفتوحة، تقدر تكتب وتنفذ ال subcommands الباقية براحتك مثل إنشاء بارتيشن، حذف بارتيشن، وغيرها).

بشكل افتراضي، تقوم أداة **parted** بعرض جميع الأحجام والمساحات باستخدام قوى العشرة مثل KB و MB و GB

➤ By default, **parted** displays all the sizes in powers of 10 (KB, MB, GB).

You can change that default with the **unit** subcommand which accepts the following parameters:

يمكنك تغيير هذا العرض الافتراضي باستخدام الـ subcommand المسمى **unit**، والذي يقبل معاملات **parameters** مختلفة لتحديد وحدة القياس المناسبة لك:

- **s** for sector

- **B** for byte

- **MB, GB, or TB** (powers of 10)

- استخدام الحرف **s** لعرض المساحات بوحدة السيكتور **sector**

- استخدام الحرف **B** لعرض المساحات بوحدة البايت **byte**

- استخدام **MB** أو **GB** أو **TB** لعرض المساحات بقوى العشرة المعروفة.

```
[root@host ~]# parted /dev/nvme0n1 unit s print
```

```
[root@host ~]# parted /dev/nvme0n1 unit B print
```

```
[root@host ~]# parted /dev/nvme0n1 unit MB print
```

```
[root@host ~]# parted /dev/nvme0n1 unit GB print
```

```
[root@host ~]# parted /dev/nvme0n1 unit TB print
```

- ال sector (السيكتور الواحد) عادة ما يساوي 512 بايت.
- عندما تستخدم الحرف B (أي B unit)، فهذا يعني أنك تطالب الأداة بأن تعرض وتستعرض سائز ومساحات البارتيشن بدقة شديدة مقاسة بالبايت بالكامل.

شرح الأوامر والنتيجة المطلوبة من كل أمر:

- `parted /dev/nvme0n1 unit s print`: النتيجة المرجوة هي طباعة جدول التقسيم والبارتيشنز الخاصة بالديسك `dev/nvme0n1/` مع عرض كافة المساحات والأحجام مقاسة بالسيكتور (sectors).
- `parted /dev/nvme0n1 unit B print`: النتيجة المرجوة هي طباعة معلومات وفحص البارتيشنز للديسك مع عرض المساحات والأحجام بدقة شديدة جداً مقاسة بالبايت (Bytes).
- `parted /dev/nvme0n1 unit MB print`: النتيجة المرجوة هي طباعة جدول التقسيم للديسك مع عرض المساحات والأحجام مقاسة بوحدة الميجابايت (MB).
- `parted /dev/nvme0n1 unit GB print`: النتيجة المرجوة هي طباعة وتوضيح جدول التقسيم للديسك مع عرض الأحجام مقاسة بوحدة الجيجابايت (GB).
- `parted /dev/nvme0n1 unit TB print`: النتيجة المرجوة هي طباعة خريطة التقسيم للديسك مع عرض المساحات الكبيرة مقاسة بوحدة التيرابايت (TB).

Writing the Partition Table on a New Disk

- To partition a new drive, you first have to write a disk label to it. The disk label indicates which partitioning scheme to use.
- Keep in mind that **parted** makes the changes immediately. A mistake with **parted** could definitely lead to data loss.
- As the root user, **use the following command to write an MBR disk label to a disk:**

```
[root@host ~]# parted /dev/nvme0n1 mklabel msdos
```

- To write a GPT disk label, **use the following command:**

```
[root@host ~]# parted /dev/nvme0n1 mklabel gpt
```

- The **mklabel** subcommand ^{محو جدول التقسيم القديم بالكامل} wipes the existing partition table. Only use **mklabel** when the intent is to reuse the disk without regard to the existing data. If a new label changes the partition boundaries, all data in existing file systems will become inaccessible.

Creating MBR Partitions

- 1- Specify the disk device to create the partition on.

```
[root@host ~]# parted /dev/vdb
```

- As the root user, execute the **parted** command and specify the disk device name as an argument. This starts the **parted** command in interactive mode and displays a command prompt.
- 2- Use the **mkpart** subcommand to create a new primary or extended partition.

```
(parted) mkpart  
Partition type? primary/extended? primary
```

- For situations where you need more than four partitions on an MBR-partitioned disk, create three primary partitions and one extended partition. This extended partition serves as a container within which you can create multiple logical partitions.

Creating MBR Partitions

هنا تقوم بكتابة نوع **file system** الذي ستستخدمه لاحقاً، مثل **ext4** أو **xfs**.

- 3- Indicate the file-system type that you want to create on the partition, such as xfs or ext4.
This does not create the file system on the partition; it is only an indication of the partition type.

```
File system type? [ext2]? xfs
```

- To get the list of the supported file-system types, use the following command:

```
[root@host ~]# parted /dev/vdb help mkpart
```

- 4- Specify the sector on the disk that the new partition starts on.

```
Start? 2048s
```

OR Start? 1GB

Notice the **s** suffix to provide the value in sectors. You can also use the **MB**, **GB**, or **TB** suffixes.
If you do not provide a suffix, **MB** is the default.

إذا لم تقم بكتابة أي لاحقة نهائياً مع الرقم، فإن الـ **MB** سيكون هو القيمة الافتراضية التي تعتمد عليها الأداة.

Creating MBR Partitions

ترتيب العمليات الصح هو: تخرج بـ quit <تنفذ udevadm settle للتأكد من ظهور البارتيشن في نظام التشغيل> تعمل فورمات للبارتيشن (إنشاء نظام ملفات) < تنشئ مجلد الـ Mount > تعمل mount فعلي < أخيراً تحدث ملف /etc/fstab عشان يثبت معاك دائم مع كل إعادة تشغيل.

5- Specify the disk sector where the new partition should end.

```
End? 1000MB
```

- As soon as you provide the end position, **parted** updates the partition table on the disk with the new partition details.

6- Exit **parted**.

```
(parted) quit
```

```
Information: You may need to update /etc/fstab.
```

7- Run the **udevadm settle** command. This command waits for the system to detect the new partition and to create the associated device file under the **/dev** directory. It only returns when it is done.

```
[root@host ~]# udevadm settle
```

للتأكد من ظهور البارتيشن في نظام التشغيل (لأن أحياناً بيتسجل في جدول التقسيم ولكن الكيرنل بتاع اللينكس بياخد جزء من الثانية على ما يحس بيه، فده بيخليك تضمن إن البارتيشن ظاهر فعلياً وجاهز)

Creating MBR Partitions

- As an alternative to the interactive mode, you can also create the partition as follows:

```
[root@host ~]# parted /dev/vdb mkpart primary xfs 2048s 1000MB
```

- As an alternative to the interactive mode, you can also create the partition as follows:

```
parted /dev/nvme0n2 <<END  
mkpart primary 0% 10%  
quit  
END
```

طريقة الـ Here Document في الـ Bash Scripting

- هي تقنية تعتمد على استخدام علامتي الأصغر من (<<) تُعرّف بأنها تقنية تسمح للـ Bash إنه يمرر عدة أسطر من المدخلات النصية (التي تمثل عدة أوامر) إلى أداة مثل parted كأنك تكتبها بيدك بالضبط في الوضع التفاعلي.
- كلمة END (أو أي كلمة اختيارية أخرى تحددها) تُستخدم كعلامة لتحديد نقطة البداية والنهاية لكتلة المدخلات.

Creating GPT Partitions

- The GTP scheme also uses the **parted** command to create new partitions.

1- Specify the disk device to create the partition on.

```
[root@host ~]# parted /dev/vdb
```

2- Use the **mkpart** subcommand to start creating the new partition.

- With the GPT scheme, each partition is given a name.

```
(parted) mkpart  
Partition name? []? usersdata
```

3- Indicate the file system type that you want to create on the partition.

```
File system type? [ext2]? xfs
```

Creating GPT Partitions

4- Specify the sector on the disk that the new partition starts on.

Start? **2048s**

5- Specify the disk sector where the new partition should end.

End? **1000MB**

6- Exit **parted**.

(parted) **quit**

Information: You may need to update /etc/fstab.

7- Run the **udevadm settle** command.

```
[root@host ~]# udevadm settle
```

- As an alternative to the interactive mode, you can also create the partition as follows:

```
[root@host ~]# parted /dev/vdb mkpart primary xfs 2048s 1000MB
```

Create Partition Named Primary

Deleting Partitions

➤ The following steps apply for both the MBR and GPT partitioning schemes.

1- Specify the disk that contains the partition to be removed.

```
[root@host ~]# parted /dev/vdb
```

2- Identify the partition number of the partition to delete.

```
(parted) print
```

3- Delete the partition.

```
(parted) rm1
```

➤ The **rm** subcommand immediately deletes the partition from the partition table on the disk.

4- Exit **parted**.

```
(parted) quit
```

أوامر العرض في الصورة الأولى (`lsblk`):

- الأمر `lsblk` : يعرض لك الـ `disks` الموجودة على النظام وكل الـ `partitions` بداخلتها مع الـ `mountpoints` الخاصة بها.
- بخصوص تسمية الأقراص:
 - هاردات الـ `SATA SSD` تبدأ بالرمز `sda` للأول، `sdb` للثاني، وهكذا (`...sda1, sda2... sdb1, sdb2`).
 - هاردات الـ `HDD` التقليدية تظهر بـ `hda` , `hdb` (`...hda1, hda2... hdb1, hdb2`).
 - الأقراص الحديثة مثل `NVMe` تظهر بالشكل `nvme0n1` وتكون البارتيشنات تابعة لها مثل `nvme0n1p1`.
- الأمر `lsblk -p` : يعرض لك الـ `full path` (المسار الكامل) للبارتيشن، وكلها تقع تحت مجلد الـ `dev/`.
- الأمر `lsblk -pf` : يعرض لك الـ `partitions` بجوار الـ `file system` المستخدم عليها.

```
parted /dev/nvme0n1 print
```

وهو الأمر الذي يفتحه المبرمج/المدرّب لعرض تفاصيل التقسيم على الهارد المختار.

○ نوع الديسك: يظهر في خانة ال Model أنه هارد افتراضي تم إنشاؤه على **VMware**.

○ نوع جدول التقسيم (**Partition Table**): تجده مكتوباً **msdos** ، وهذا معناه أن الديسك يستخدم نظام ال **MBR schema**.

○ تحديد ال **Primary** وال **Extended**: أنت من تقوم بتحديد نوع البارتيشن أثناء إنشائه (سواء كان **primary** أو **extended**) بناءً على حاجتك.

○ معنى ال **Flags**: ال **Flags** (مثل **boot** أو **lba**) هي علامات برمجية تقرأها البيئة أو ال BIOS/UEFI لتوجيه النظام (مثل تحديد بارتيشن الإقلاع الأساسي)، والبارتيشن الخالي منها هو بارتيشن تخزيني عادي.

○ أمر تغيير وحدة القياس إلى السكتور (**unit s print**):

```
parted /dev/nvme0n1 unit s print
```

هذا الأمر يُستخدم لتغيير وحدة عرض القياس في جدول التقسيم إلى **Sectors** (السكتورات)، مما يتيح لك رؤية بدايات ونهايات الأقسام بدقة متناهية على مستوى القطاعات الفيزيائية للديسك.

○ أمر تغيير وحدة القياس إلى الميجابايت (**unit MB print**):

```
parted /dev/nvme0n1 unit MB print
```

هذا الأمر يُستخدم لتغيير وحدة عرض القياس في جدول التقسيم إلى **Megabytes** (الميجابايت)، لتسهيل قراءة أحجام الأقسام والمساحات التخزينية بالوحدات العشرية المتعارف عليها.

- تنفيذ أمر: `parted /dev/nvme0n1` والدخول في الجلسة التفاعلية الخاصة بالهارد المحدد لعرض وإدارة التقسيمات.
- تنفيذ أمر: `print` لطباعة وعرض تفاصيل البارتيشنات الحالية الموجودة على الديسك.
- تنفيذ أمر: `help mkpart`
- ملحوظة: يتم استخدام أمر `help` مع `mkpart` لمعرفة خياراته والـ `arguments` المتاحة وأنواع الـ `partition-types` وأنظمة الملفات المدعومة.
- تطبيق إنشاء بارتيشن جديد عبر الـ `interactive prompts` (تحديد نوعه `logical`، نوع الـ `file system` مثل `ext4`، والـ `Start` والـ `End`).
- تنفيذ أمر: `print`
- تنفيذ أمر: `rm 6`
- تنفيذ أمر: `print`
- تنفيذ أمر: `quit`
- اذهب إلى إعدادات برنامج **VMware** وأضف هارد ديस्क جديد يكون من نوع `nvme` (مع ملاحظة أن تكون قد قمت بعمل `Power Off` للـ `VMware` قبل تنفيذ هذه الخطوة لتتمكن من إضافة الهارد بنجاح)، وأعطه مساحة 10 جيجابايت.
- تنفيذ أمر: `lsblk`

```

o تنفيذ أمر: mklabel msdos
o تنفيذ أمر: print
o تنفيذ أمر: mkpart primary xfs 0% 20%
o تنفيذ أمر: p
o تنفيذ أمر: mkpart primary ext4 20% 40%
o تنفيذ أمر: print
o تنفيذ أمر: % unit
o تنفيذ أمر: print
o تنفيذ أمر: mkpart primary ext4 40% 60%
o تنفيذ أمر: mkpart primary ext4 60% 80%
o تنفيذ أمر: print
o تنفيذ أمر: mkpart primary ext4 80% 100%
o تنفيذ أمر: rm 4
o تنفيذ أمر: print

```

```

o تنفيذ أمر: rm 4
o تنفيذ أمر: print
o تنفيذ أمر: mkpart primary extended 60% 100%
o تنفيذ أمر: mkpart logical ext3 60% 80%
o تنفيذ أمر: print
o تنفيذ أمر: mkpart logical ext2 80% 100%
o تنفيذ أمر: unit MB
o تنفيذ أمر: print
o تنفيذ أمر: quit

```

ملحوظة هامة: لو لاحظت رغم أننا مدين أول بارتيشن أن يبدأ من 0% إلا أنه بدأ وحجز مساحة (حوالي 1 ميجابايت أو ما يقاربها بداية من 1.05 MB وعمل لها Alocate لتخزين partition table الخاص بال-MBR)

Bash



```
parted /dev/nvme0n2 rm 6
parted /dev/nvme0n2 print
parted /dev/nvme0n2 mkpart logical xfs 80% 100%
parted /dev/nvme0n2 print
parted /dev/nvme0n2 <<EOT
rm 1
rm 2
rm 3
rm 4
quit
EOT
vi parted.sh
bash parted.sh
parted /dev/nvme0n2 print
```

Content Of Script

```
#!/bin/bash
```

```
parted /dev/nvme0n2 <<END
mkpart primary 0% 20%
mkpart primary 20% 40%
mkpart primary 40% 60%
mkpart extended 60% 100%
mkpart logical 60% 80%
mkpart logical 80% 100%
quit
END
```

Bash



```
parted /dev/nvme0n2 mklabel gpt
parted /dev/nvme0n2 print
parted /dev/nvme0n2
mkpart
nvme0n2p1
xfs
0%
50%
p
qiut
parted /dev/nvme0n2
p
mkpart nvme0n2p2 ext4 50% 100%
p
rm 1
rm 2
p
qiut
vi parted.sh
bash parted.sh
parted /dev/nvme0n2 print
lsblk -pf
```

ملاحظة سريعة عن `fdisk` و `gdisk` :
أداة `fdisk` مصممة أساساً للتعامل مع الـ MBR schema (ويمكن استخدامها مع GPT ولكن بشكل محدود)، بينما أداة `gdisk` مصممة خصيصاً للتعامل مع الـ GPT partition tables.

أوامر تفاعلية داخل `fdisk` (شرح الأوامر الظاهرة في الصور):

عند كتابة أمر الدخول على الديسك:



Bash

```
fdisk /dev/nvme0n2
```

تدخل في الـ Interactive Session، ويدخل التغييرات في الذاكرة أولاً ولا يتم حفظها على الديسك إلا باستخدام أمر الحفظ النهائي.

عند الضغط على الحرف `m` (لطلب المساعدة `m for help`) تظهر القائمة التالية مقسمة إلى أجزاء رئيسية:

Generic (الأوامر العامة للبارتيشنات):

- `d` : لحذف بارتيشن (delete a partition).
- `f` : لعرض المساحات الفارغة غير المقسمة (list free unpartitioned space).
- `l` : لعرض أنواع البارتيشنات المعروفة (list known partition types).
- `n` : لإنشاء بارتيشن جديد (add a new partition).
- `p` : لطباعة وعرض جدول البارتيشنات الحالي (print the partition table).
- `t` : لتغيير نوع البارتيشن (change a partition type).
- `v` : للتحقق من سلامة جدول البارتيشنات (verify the partition table).
- `i` : لعرض معلومات تفصيلية عن بارتيشن معين (print information about a partition).

Misc (أوامر متفرقة):

- m : لطباعة هذه القائمة التوجيهية (print this menu).
- x : لخيارات متخصصة للمحترفين وخبراء النظام (extra functionality - experts only).

Script (السكربتات):

- I : تحميل تخطيط السكربت: تقوم بتجهيز ملف نصي خارجي يحتوي على أبعاد وتقسيمات الديسك، وعند استخدام خيار التحميل تقوم الأداة بقراءة هذا الملف وتطبيق التقسيمات دفعة واحدة.
- O : تصدير تخطيط الديسك الحالي: يمكنك أخذ نسخة من إعدادات وتقسيمات الديسك الحالية وحفظها داخل ملف خارجي كمرجع أو سكربت لإعادة استخدامه لاحقاً.

Save & Exit (الحفظ والخروج):

- w : لكتابة التغييرات وحفظها على الديسك ثم الخروج (write table to disk and exit). كل التغييرات المؤقتة لا تُحفظ نهائياً إلا بهذه الخطوة وتظل ثابتة حتى بعد إعادة التشغيل.
- q : للخروج دون حفظ أي تغييرات تم إجراؤها (quit without saving changes).

Create a new label (إنشاء جدول أقسام/اسكيما جديدة):

- g : لإنشاء جدول أقسام جديد من نوع GPT فارغ (create a new empty GPT partition table).
- G : لإنشاء جدول أقسام SGI (IRIX).
- o : لإنشاء جدول أقسام جديد من نوع MBR (DOS) فارغ (تغيير الاسكيما إلى MBR).
- s : لإنشاء جدول أقسام Sun فارغ.

خطوات التنفيذ بالتفصيل جوه ال Interactive Session للـ fdisk بناءً على الصور اللي بعثتها:

- طباعة الجدول الحالي للديسك:
- اكتب الحرف `p` لعرض حالة الأقسام الحالية على الديسك.
- تغيير ال Disklabel إلى DOS (MBR):
- اكتب الحرف `n` لإنشاء جدول أقسام جديد من نوع DOS (MBR). ودي الخطوة اللي حولت الاسكيما من MBR لـ GPT.
- التحقق من التغيير:
- اكتب `p` مرة أخرى للتأكد إن ال Disklabel نوعها بقى `dos`.
- إنشاء البارتيشن الأول (`dev/nvme0n2p1/`):
- اكتب الحرف `n` لإنشاء بارتيشن جديد.
- اختر نوع البارتيشن `p` (primary) أو اضغط `Enter` للإجابة الافتراضية.
- حدد رقم البارتيشن `1` (أو اضغط `Enter`).
- اختر السيكتور الأول الافتراضي (اضغط `Enter`).
- في المساحة أو الحجم (`Last sector`) اكتب `2G+` لتحديد مساحة البارتيشن بـ 2 جيجابايت.
- التحقق بعد إنشاء أول بارتيشن:
- اكتب `p` لطباعة الجدول ورؤية البارتيشن الأول (`dev/nvme0n2p1/`) بمساحة 2G ونوع `Linux 83`.
- إنشاء البارتيشن الثاني (`dev/nvme0n2p2/`):

○ إنشاء البارتيشن الثاني (/dev/nvme0n2p2):

○ اكتب الحرف n .

○ اختر النوع p ورقم البارتيشن 2 .

○ اضغط Enter للبداية الافتراضية، وفي الحجم اكتب 2G+ .

○ إنشاء البارتيشن الثالث (/dev/nvme0n2p3):

○ اكتب الحرف n .

○ اختر النوع p ورقم البارتيشن 3 .

○ اضغط Enter للبداية، وفي الحجم اكتب 2G+ .

○ التحقق بعد إنشاء الثلاث بارتيشنات:

○ اكتب p لعرض الجدول والتأكد من وجود أول ثلاث بارتيشنات أساسية (Primary).

○ إنشاء البارتيشن الرابع الممتد (Extended):

○ اكتب الحرف n .

○ في اختيار النوع (Partition type)، هيظهر لك خيار p (primary) أو e (extended)،
اختار e لعمل حاوية للبارتيشنات المنطقية المتبقية.

○ اختر رقم البارتيشن 4 (افتراضي).

○ اضغط Enter للبداية، واضغط Enter للنهاية لاختيار المساحة المتبقية بالكامل ليتم إنشاء
بارتيشن Extended بمساحة 4 جيجابايت.

○ العرض النهائي للجدول:

○ اكتب الحرف p لمراجعة وتأكيذ توزيع البارتيشنات الثلاثة الأساسية والبارتيشن الرابع ال-
Extended قبل حفظ التغييرات نهائياً.

○ إنشاء البارتيشن الخامس (Logical):

○ اكتب الحرف `n` لإنشاء بارتيشن جديد.

○ نظراً لامتلاء البارتيشنات الأساسية، ستظهر رسالة

`All primary partitions are in use` ويتم إنشاء البارتيشن كـ

`Adding logical partition 5` بشكل مباشر.

○ اضغط `Enter` للقبول بالبداية الافتراضية للسيكتور.

○ في الحجم (`Last sector`) اكتب `2G+` لتحديد مساحة البارتيشن بـ 2 جيجابايت.

○ مراجعة الجدول قبل الحفظ:

○ اكتب الحرف `p` لطباعة الجدول ورؤية البارتيشنات الخمسة (الثلاثة الأساسية، وال `Extended`

الرابع، والخامس ال `Logical`).

○ حفظ التغييرات:

○ اكتب الحرف `w` لكتابة وحفظ التغييرات نهائياً على الديسك والخروج من الجلسة التفاعلية،

وتظهر رسالة `The partition table has been altered` و `Syncing disks`.

○ التحقق النهائي:

○ اكتب الأمر التالي خارج الجلسة التأكيد والتأكد من حفظ وتطبيق التقسيمات:



Bash

```
fdisk -l /dev/nvme0n2
```

(أو بالطريقة الشاملة للتحقق عبر `lsblk -pf`) للتأكد أن كل البارتيشنات ظهرت وتم حفظها بنجاح.

خطوات التعامل مع أمر `gdisk` وتوليد وعرض وحذف البارتيشنات بالتفصيل بناءً على الصور المرفقة:

- رسالة الفحص والتحويل التلقائي للـ MBR القديم:

- أول ما فتحت أمر `gdisk /dev/nvme0n2` ، عمل سكان للديسك وظهرت الرسالة الشهيرة:

```
Found invalid GPT and valid MBR; converting MBR to GPT format in memory
```

- المعنى هنا إن الديسك كان عليه جدول أقسام قديم نوعه MBR، والأداة `gdisk` الخاصة بـ GPT نبهتك إنها هتقوم بتحويله مؤقتاً للـ GPT في الذاكرة (وهي عملية مدمرة للبيانات القديمة POTENTIALLY DESTRUCTIVE لو مش محتاجها)، وطلبت منك تدوس `q` لو مش عايز تكمل التحويل.

- استعراض قائمة الأوامر المتاحة (?) :

- بدلاً من حرف `m` المستخدم في `fdisk` التقليدي، بتستخدم علامة الاستفهام `?` مع `gdisk` عشان تظهر لك قائمة المساعدة والـ options المتاحة بالكامل (زي `o` لإنشاء جدول GPT جديد، `p` للطباعة، `n` للإنشاء، `d` للحذف، `w` للحفظ، وهكذا).

- إنشاء جدول أقسام GPT جديد (o) :

- كتابة الحرف `o` بيعمل إنشاء لجدول أقسام فارغ جديد من نوع (GPT) GUID.

- بيظهر لك تحذير يسألك (Y/N) Proceed?: وبتكتب `Y` عشان توافق.

- إنشاء بارتيشن جديد (n) :

○ إنشاء بارتيشن جديد (n):

○ اكتب الحرف n لإنشاء بارتيشن جديد.

○ يبدأ يسألك عن رقم البارتيشن (المتاح عندك من 1 لحد 128 بارتيشن كحد أقصى بفضل دعم GPT العالي، عكس MBR المحدود بـ 4 بارتيشنات أساسية فقط).

○ بتضغط Enter لاختيار البارتيشن رقم 1 افتراضياً.

○ بتحدد السيكتور الأول والسيكتور الأخير أو المساحة المطلوبة (أو بتضغط Enter لأخذ المساحة الكاملة للديسك).

○ بيحدد نوع ال Hex code أو ال GUID الافتراضي للينكس (8300) ويتم إنشاء البارتيشن بنجاح.

○ طباعة واستعراض الجدول (p):

○ اكتب الحرف p لعرض تفاصيل الديسك وجدول الأقسام.

○ بتلاحظ إن Partition table holds up to 128 entries (بمعنى أن جدول أقسام GPT يدعم حتى 128 بارتيشن كحد أقصى)، وبتشوف قدامك البارتيشن الأول (dev/nvme0n2p1/) ومساحته الكاملة ونوعه Linux filesystem 8300.

○ حذف البارتيشن (d):

○ اكتب الحرف d لحذف البارتيشن.

○ بيطلب منك تحديد رقم البارتيشن المراد حذفه، وبتكتب 1 ليتم حذفه مباشرة من الذاكرة قبل حفظ التغييرات نهائياً.

Creating File Systems

التقسيم والتهيئة التلقائية **Auto** عندما يعتمد التثبيت أو الإعداد على خيار التثبيت التلقائي
Auto-partitioning، فإن برنامج التثبيت الخاص بنظام Red Hat Enterprise Linux يعتمد نظام **XFS** كخيار افتراضي by default بالتهيئة الأقراص وتجهيزها دون أن تضطر لتحديد ذلك يدوية.

- After the creation of a block device, the next step is to add a file system to it. Red Hat Enterprise Linux supports many different file system types, but two common ones are **XFS** and **ext4**, the installer for Red Hat Enterprise Linux, uses **XFS** by default.
- As root, use the **mkfs.xfs** command to apply an XFS file system to a block device. For ext4, use **mkfs.ext4**.

```
[root@host ~]# mkfs.xfs /dev/vdb1
```

Mounting File Systems

- After you have added the file system, the last step is to mount the file system to a directory in the directory structure. When you mount a file system onto the directory hierarchy, user can access or write files on the device.

```
[root@host ~]# mount /dev/vdb1 /mnt  
[root@host ~]# cat /etc/fstab
```

- When you add or remove an entry in the **/etc/fstab file**, run the **systemctl daemon-reload** command, or reboot the server, for systemd to register the new configuration.

○ أمر الفحص الشامل (`lsblk -pf`):

○ استخدمت الأداة لعرض جميع الأجهزة والبارتيشنات بمساراتها الكاملة مع معرفاتها الفريدة (`UUID`) وأنظمة الملفات الحالية (`FSTYPE`) وأماكن الـ `MOUNTPOINTS` للتأكد من حالة الديسكات (`dev/nvme0n1/` و `dev/nvme0n2/`).

○ استعراض أدوات الـ التهيئة (`mkfs` مع زر Tab):

○ عند كتابة `mkfs` والضغط مرتين على زر الـ Tab ، تظهر قائمة الأدوات المتاحة للتهيئة مثل (`mkfs.vfat` , `mkfs.minix` , `mkfs.xfs` , `mkfs.ext4` , `mkfs.ext3` , `mkfs.ext2` , `mkfs.msdos` , `mkfs.cramfs`).

○ تطبيق نظام الملفات باستخدام الخيار `t-`:

○ تم استخدام أمر `mkfs -t xfs /dev/nvme0n2p1` لتحديد نوع نظام الملفات يدوياً عبر الخيار `t-` وتهيئة البارتیشن الأول بنجاح، وظهرت تفاصيل بيانات الـ `XFS metadata` الخاصة به.

○ محاولة تهيئة نظام غير مدعوم أو غير متوفر (`ntfs`):

○ عند تجربة الأمر `mkfs -t ntfs /dev/nvme0n2p3` ، ظهرت رسالة خطأ تفيد بأن الأداة غير موجودة (`mkfs: failed to execute mkfs.ntfs: No such file or directory`) نظراً لأن نظام الملفات هذا غير مثبت أو غير مدعوم افتراضياً في بيئة `RHEL` الأساسية بدون حزم إضافية.

○ التحقق من المساعدة والأنواع المدعومة: استخدم الأمر `parted /dev/nvme0n2 help mkpart` لتلقي نظرة فاحصة ومعرفة الأنواع والخيارات المدعومة الخاصة بإنشاء التقسيمات وأنظمة الملفات.

```
df -h
mkdir /data2
mount /dev/nvme0n2p1 /data2
df -h
lsblk -pf أو blkid
mkdir /data3
mount UUID="<uuid>" /data3
vim /etc/fstab
mount -a
systemctl daemon-reload
umount /data3
```

بعد تنفيذ أمر `umount` ، افتح ملف `/etc/fstab` واحذف السطر الخاص بالبارتيشن نهائياً .

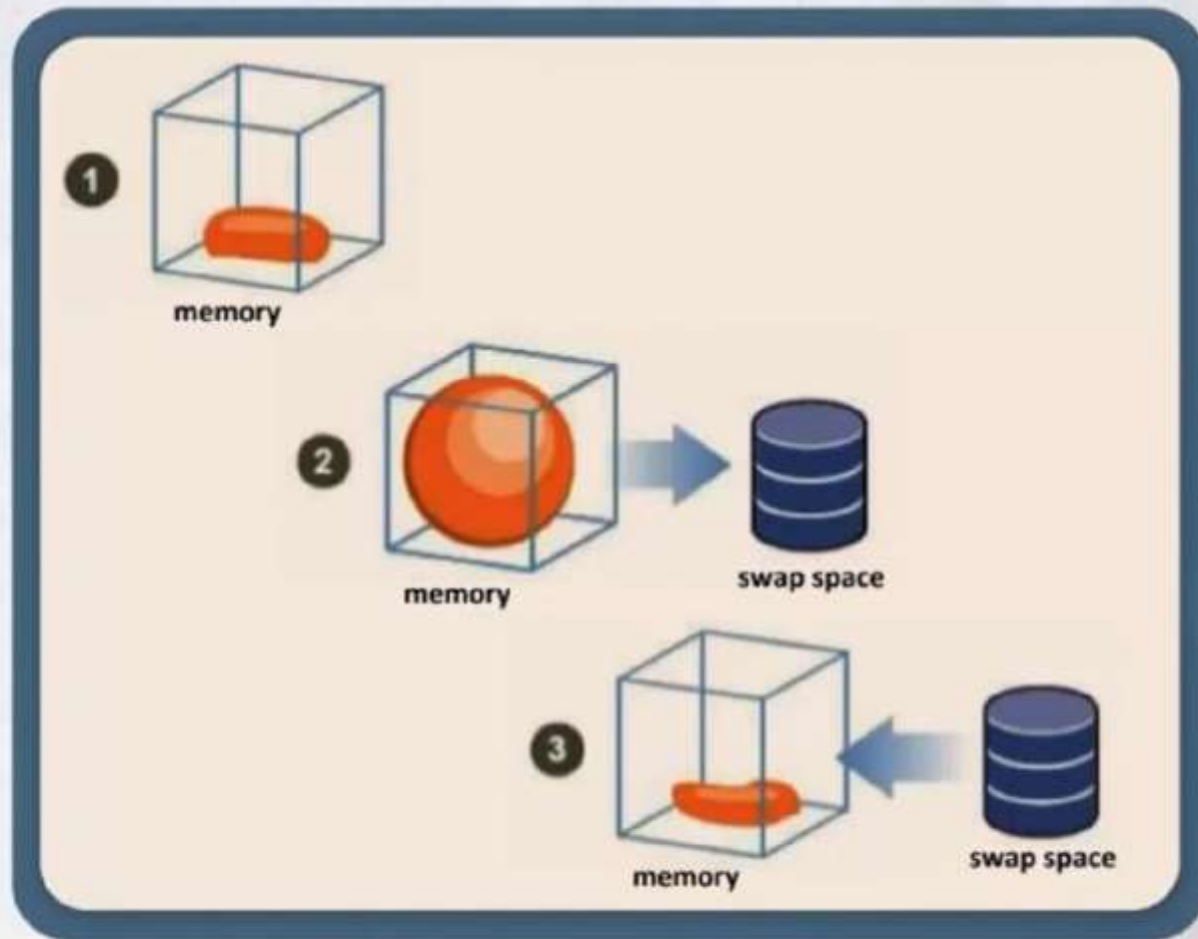
شرح الأوامر الأساسية:

○ أمر `mount -a` والتحقق من الأخطاء:

○ يُستخدم هذا الأمر لربط جميع أنظمة الملفات غير المرتبطة والمذكورة داخل ملف `/etc/fstab` .

○ أهميته الكبرى تكمن في أنه يعمل بمثابة "اختبار فوري"؛ فإذا كان هناك أي خطأ إملائي، أو مسار غير صحيح، أو `UUID` غير مضبوط داخل ملف `fstab` ، سيقوم الأمر بإظهار الخطأ فوراً على الشاشة قبل أن تقوم بعمل إعادة تشغيل (`reboot`) للسيرفر، مما يمنع وقوع النظام في مشكلة عدم الإقلاع (`Emergency Mode`).

Managing swap space



إدارة مساحة التبادل (Swap Space Management):

- **ما هي الـ Swap Space؟** هي مساحة يتم تخصيصها على القرص الصلب (Hard Disk) ليستخدمها نظام التشغيل كذاكرة افتراضية (Virtual Memory) عندما تمتلئ الذاكرة العشوائية الفيزيائية (Physical RAM) بالكامل.
- **تمديد الذاكرة الافتراضية (Virtual Memory Extension):** تساعدك على زيادة الحجم الإجمالي للذاكرة عبر دمج حجم الـ ram الفعلية مع مساحة السواب؛ فمثلاً لو كانت الـ RAM بحجم 8 جيجا ووجدت 2 جيجا سواب، سيصبح إجمالي الذاكرة الظاهرية للنظام 10 جيجابايت، مما يحسن من قابلية التوسع (Scalability).
- **الوقاية من انهيار النظام (Out of Memory Error):** عند امتلاء الـ RAM بالكامل ومحاولة تطبيق جديد العمل بدون توفر سواب، يصبح النظام بطيئاً وغير متجاوب (Unresponsive) وقد يتعرض للكراش بسبب خطأ شهير يسمى Out of Memory.
- **آلية عمل الترحيل (Swapping Pages):** يبحث النظام داخل الـ ram عن الصفحات غير النشطة أو الخاملة (Inactive/Idle Pages) وينقلها مؤقتاً لتخزينها على مساحة السواب في الـ هارد ديسك، لتفريغ مساحة كافية للعمليات الجديدة وحماية استقرار النظام رغم كون الـ هارد ديسك أبطأ نسبياً من الـ ram.
- **دعم وضع الإسبات (Hybernation Support):** تتيح ميزة الـ Hibernation حفظ كافة محتويات الذاكرة الحالية ونقلها بالكامل إلى مساحة السواب على الـ هارد عند إيقاف تشغيل الجهاز، بحيث تستعيد كافة تطبيقاتك وبياناتك كما تركتها تماماً فور إعادة تشغيل النظام، وهو أمر لا يمكن دعمه بدون وجود مساحة السواب.

شرح الصورة التوضيحية (Managing swap space):

- **الحالة الأولى (1):** تمثل الـ RAM في حالتها الطبيعية (Normal Utilization)، حيث تكون محملة بشكل عادي وغير ممتلئة.
- **الحالة الثانية (2):** تظهر الـ ram وقد امتلأت واستُخدمت بنسبة كبيرة جداً، وهنا يتدخل النظام لنقل وحفظ البيانات أو الصفحات الخاملة مؤقتاً إلى مساحة الـ (Swap Space) على الـ هارد ديسك لتفريغ مساحة للعمليات الجديدة.
- **الحالة الثالثة (3):** توضح العكس؛ فعندما يقل الضغط وتعود الـ RAM لحالتها الطبيعية، يقوم النظام بسحب واستعادة البيانات التي كانت مخزنة على مساحة السواب وإرجاعها مرة أخرى إلى الذاكرة العشوائية لسرعة وسهولة التعامل معها.

Introducing SWAP Space Concepts

- A *swap space* is an area of a disk under the control of the Linux kernel memory management subsystem. The kernel uses swap space to supplement the system RAM by holding inactive pages of memory. The combined system RAM plus swap space is called *virtual memory*.
- When the memory usage on a system exceeds a defined limit, the kernel searches through RAM looking for idle memory pages assigned to processes. The kernel writes the idle pages to the swap area and reassigns the RAM pages to other processes. If a program requires access to a page on disk, the kernel locates another idle page of memory, writes it to disk, then recalls the needed page from the swap area.
- Because swap areas reside on disk, swap is slow when compared with RAM. While it is used to augment system RAM, you should not consider swap space as a sustainable solution for insufficient RAM for your workload.

تعريف مساحة التبادل والذاكرة الافتراضية:

- مساحة الـ swap space هي عبارة عن منطقة من القرص تحت تحكم وحدة إدارة الذاكرة في كيرنل لينكس (Linux kernel memory management subsystem).
- يستخدم الكيرنل الـ swap space لدعم وتكملة رام النظام (system RAM) من خلال الاحتفاظ بالـ inactive pages of memory.
- يطلق على مجموع system RAM مضافاً إليها الـ swap space اسم الـ virtual memory.

آلية عمل الكيرنل عند امتلاء الذاكرة:

- عندما يتجاوز استخدام الذاكرة على النظام حداً معيناً (defined limit)، يبحث الكيرنل داخل الـ RAM عن الـ idle memory pages المخصصة للـ processes.
- يكتب الكيرنل الـ idle pages إلى الـ swap area ويعيد تعيين الـ RAM pages للـ processes أخرى.
- إذا طلب برنامج الوصول إلى صفحة على القرص (page on disk)، يحدد الكيرنل مكان الـ idle page أخرى في الذاكرة، ويكتبها إلى القرص، ثم يستدعي الصفحة المطلوبة من الـ swap area.

الأداء والاعتمادية:

- نظراً لأن الـ swap areas توجد على القرص (on disk)، فإن الـ swap يكون بطيئاً (slow) مقارنةً بالـ RAM.
- بينما يتم استخدامه لزيادة رام النظام (augment system RAM)، يجب ألا تعتبر الـ swap space حلاً مستداماً (sustainable solution) لنقص الـ RAM بالنسبة للـ workload الخاص بك.

Sizing the SWAP Space

- Administrators should size the swap space based on the memory workload on the system. Application vendors sometimes provide recommendations on that subject. **The following table provides some guidance based on the total amount of physical memory:**

RAM	SWAP SPACE
<i>2 GiB or less</i>	Twice the RAM
<i>Between 2 GiB and 8 GiB</i>	Same as RAM
<i>Between 8 GiB and 64 GiB</i>	At least 4 GiB
<i>More than 64 GiB</i>	At least 4 GiB

SWAP Partition

```
[root@host ~]# parted /dev/vdb
```

```
(parted) mkpart
```

```
Partition name? []? Swap1
```

```
File system type? [ext2]? linux-swap
```

```
Start? 1001MB
```

```
End? 1257MB
```

```
(parted) print
```

```
(parted) quit
```

```
[root@host ~]# udevadm settle
```

Formatting the Device

```
[root@host ~]# mkswap /dev/vdb2
```

Activating a SWAP Space

```
[root@host ~]# swapon /dev/vdb2
```

SWAP File

```
# dd if=/dev/random of=/swap_file bs=1M count=1024
```

```
# fallocate -l 1G /swap_file
```

```
# mkswap /swap_file (Format the device)
```

```
# swapon /swap_file (Activate a swap space)
```

```
# swapon -a (activate all swap spaces listed in the  
/etc/fstab file)
```

```
# vi /etc/fstab (Persistently activate swap space)
```

```
# systemctl daemon-reload
```

```
# free -m (show the swap file system)
```

```
# swapoff /swap_file (deactivate a swap space)
```

هناك طريقتان رئيسيتان لإنشاء وإدارة مساحة التبادل (SWAP Space) في نظام لينكس:

1. الطريقة الأولى: SWAP Partition

- يتم اقتطاع بارتيشن كامل وتخصيصه للـ SWAP باستخدام أدوات التقسيم المختلفة مثل `fdisk` أو `gdisk` أو `parted`.
- **ملحوظة هامة:** لضمان استمرار عمل الـ SWAP بشكل دائم بعد إعادة تشغيل النظام (`reboot`), يجب تسجيل البيانات والريكورد الخاص بالبارتيشن داخل ملف `/etc/fstab`.

متى نلجأ للطريقة الثانية (SWAP File)؟

عند استخدام نظام التقسيم MBR والوصول للحد الأقصى المسموح به من البارتيشنات الأساسية (4 Primary Partitions) وعدم القدرة على اقتطاع بارتيشن جديد، يتم اللجوء لإنشاء ملف سواب (SWAP File).

2. الطريقة الثانية: SWAP File

تعتمد هذه الطريقة على إنشاء ملف بمساحة محددة على القرص الصلب، ويتعامل معه الكيرنل تماماً كأنه بارتيشن حقيقي.

- طرق إنشاء وتحديد حجم ملف الـ SWAP:

1. أمر `dd`:



Bash

```
dd if=/dev/random of=/swap_file bs=1M count=1024
```

الشرح: يقوم هذا الأمر بقراءة البيانات المولدة من المصدر (`if=/dev/random`) ويستمر بتعبئتها وكتابتها داخل الملف النهائي (`of=/swap_file`) بحجم بلوك يبلغ 1 ميجابايت (`bs=1M`) وبتكرار 1024 مرة (`count=1024`) ليصبح إجمالي حجم الملف 1 جيجابايت.



Bash

```
fallocate -l 1G /swap_file
```

الشرح: أداة أسرع لحجز المساحة المطلوبة (1G) وتخصيصها فوراً على القرص للملف دون الحاجة لكتابة بيانات تدريجية.

التعامل مع الملف بعد إنشائه (كأنه بارتيشن عادي):

- تهيئة الملف بنظام سواب: `mkswap /swap_file`
- تفعيل مساحة السواب: `swapon /swap_file`
- تفعيل كافة مساحات السواب المذكورة بالـ `fstab`: `swapon -a`
- تسجيل الملف ليكون دائماً: إضافة السطر الخاص بالملف داخل `vi /etc/fstab`
- تحديث تكوينات النظام: `systemctl daemon-reload`
- إلغاء تفعيل مساحة السواب: `swapoff /swap_file`

ملحوظة أمر الفحص (`free -m`):

يُستخدم خيار الخيار `m-` مع الأمر `free` لعرض حجم واستخدامات الذاكرة العشوائية (RAM) ومساحة الـ SWAP بوحدّة الميجابايت (Megabytes).

`free -m`

- شرح الأمر: يعرض حالة الذاكرة العشوائية (RAM) ومساحة التبادل (Swap) بوحدة الميجابايت. عند زيادة الضغط وامتلاء الـ RAM، يقوم الكيرنل بأخذ البيانات أو الـ inactive pages الموجودة في الـ ram ويعمل لها save في مساحة الـ swap لتفريغ مساحة للعمليات الجديدة.

`lsblk -p`

- شرح الأمر: يعرض الهاردوير والأقراص والبارتيشنات المرتبطة بالنظام بشكل تفصيلي ومساراتها الكاملة (full paths).

`parted /dev/nvme0n2 mklabel gpt`

- شرح الأمر: يُستخدم أداة parted لإنشاء جدول تقسيم جديد من نوع GPT على القرص المختار.
- رسالة التنبيه: تظهر هذه الرسالة لأن تعديل جدول التقسيم أو مسح الهارد قد يؤثر على البارتيشنات القديمة المسجلة، وللتذكير بضرورة تحديث ملف `etc/fstab/` إذا تم تغيير أي UUID أو مسار خاص بالبارتيشنات المعنية.

التطبيق العملي داخل أداة الـ parted (الخطوات التفاعلية):

- أمر `mkpart` : لبدء إنشاء بارتيشن جديد.
- تحديد اسم البارتيشن (?Partition name): تم إدخال الاسم `Swap1`.
- تحديد نوع نظام الملفات (?File system type): تم اختياره ليصبح `linux-swap`.
- تحديد نقطة البداية (?Start): تم ضبطها عند 0% من المساحة المتاحة.
- تحديد نقطة النهاية (?End): تم ضبطها عند 10% لتحديد مساحة البارتيشن المطلوبة.
- أمر الطباعة (p أو print): لعرض تفاصيل البارتيشنات التي تم إنشاؤها والتأكد من حجمها ونوعها (`linux-swap`).
- أمر الخروج (quit): لحفظ التغييرات والخروج من البيئة التفاعلية للأداة.

```
lsblk -pf
```

- **شرح الأمر:** يعرض البارتيشنات مع ملفات الأنظمة الخاصة بها (FSTYPE) وقيم الـ (UUID) والـ mountpoints للمتابعة الدقيقة.

```
mkswap /dev/nvme0n2p1
```

- **شرح الأمر:** يقوم بتهيئة البارتيشن الجديد وتجهيزه ليكون صالحاً للعمل كنظام ملفات تبادلي (swap filesystem) وتوليد UUID خاص به.

```
free -m
```

- **شرح الأمر:** فحص حجم الـ swap الكلي قبل تفعيل البارتيشن الجديد (يظهر في الصورة بمساحة 1023 ميجا).

```
swapon /dev/nvme0n2p1
```

- **شرح الأمر:** تفعيل مساحة الـ swap الجديدة المضافة على البارتيشن لتبدأ بالعمل فوراً، وعند إعادة تنفيذ `free -m` بعدها ستلاحظ زيادة الحجم الإجمالي للـ swap (ليصبح 2046 ميجا).

```
swapoff /dev/nvme0n2p1
```

- **شرح الأمر:** إلغاء تفعيل مساحة الـ swap الخاصة بالبارتيشن المؤقت وإخراجها من الخدمة، مما يؤدي لانخفاض مساحة الـ swap الإجمالية مرة أخرى عند فحصها بـ `free -m`.

تطبيق عملي على إنشاء وإدارة ملف السواب (SWAP File):

○ `free -m`

○ **شرح الأمر:** يعرض حالة الذاكرة العشوائية (RAM) ومساحة التبادل (Swap) بوحدة الميجابايت لفحص حجم الـ Swap قبل وبعد التعديل.

○ `fallocate -l 1G /swap_file`

○ **شرح الأمر:** أمر مخصص لحجز مساحة قدرها 1 جيجابايت فوراً على القرص لإنشاء ملف السواب بشكل سريع دون كتابة بيانات فعلية بداخلها.

○ `du -sh /swap_file`

○ **شرح الأمر:** لعرض المساحة الفعلية التي يشغلها ملف `swap_file/` على القرص والتأكد من أنها أصبحت 1 جيجابايت.

○ `mkswap /swap_file`

○ **شرح الأمر:** تهيئة الملف ليصبح صالحاً للعمل كنظام ملفات تبادل (`swap filesystem`).

○ **ملاحظة أمنية هامة ورسالة التنبيه:** ظهرت رسالة تحذيرية تفيد بأن صلاحيات الملف غير آمنة (`insecure permissions 0644`)؛ لأن الصلاحيات الحالية تسمح بقراءة الملف من قبل الآخرين، وحيث أن ملف السواب يعتبر حساساً للغاية ويحتوي على بيانات ذاكرة النظام، فإنه يُشترط تعديل صلاحياته لتكون مقصورة على الـ root فقط وجعله للقراءة والكتابة للـ root فقط عبر الأمر `chmod 0600 /swap_file`.

`swapon /swap_file` ○

○ **شرح الأمر:** تفعيل ملف السواب مؤقتاً ليدخل الخدمة فوراً وترتفع مساحة الـ Swap الإجمالية للنظام عند فحصها بـ `free -m`.

`chmod 0600 /swap_file` ○

○ **شرح الأمر:** تعديل صلاحيات ملف السواب لتصبح `600` (أي صلاحية القراءة والكتابة لمالك الملف `root` فقط، ومنع أي مستخدم آخر من الوصول إليه للحفاظ على سرية وحماية بيانات الذاكرة).

`ls -l /swap_file` ○

○ **شرح الأمر:** فحص صلاحيات وخصائص الملف للتأكد من تعديلها بنجاح لتصبح `-----rw-`.

التعديل الدائم في ملف الـ `fstab`:

`vi /etc/fstab` ○

○ **شرح الأمر:** فتح ملف `etc/fstab/` لإضافة السواب ليعمل بشكل دائم مع كل إعادة تشغيل للنظام، حيث تم إضافة السطر الأخير في الملف بهذا الشكل:

`swap_file none swap defaults 0 0/`

لاحظ أننا وضعنا مكان الـ `mount point` كلمة `none` لأن ملف الـ `Swap` لا يتم عمل `Mount` له على مجلد تقليدي.

الطريقة الثانية باستخدام أمر `dd` (إنشاء ملف سواب مع التعبئة):

```
dd if=/dev/zero of=/swap_file1 bs=1M count=1024
```

شرح الأمر: إنشاء ملف سواب جديد باسم `swap_file1/` باستخدام أمر `dd` ، والفرق هنا بينه وبين `fallocate` هو أن أمر `dd` يقوم بملء المساحة المحددة فعلياً ببيانات صفرية (`dev/zero/`) كتجميع بلوكات (`bs=1M` بعدد `1024` مرة ليصبح الإجمالي 1 جيجابايت)، مما يجعله أبطأ قليلاً في الإنشاء مقارنة بـ `fallocate` ولكنه يضمن حجز المساحة فعلياً بشكل كامل.

```
du -sh /swap_file1
```

شرح الأمر: التحقق من حجم الملف الناتج والتأكد من أنه بلغ 1 جيجابايت.

```
mkswap /swap_file1
```

شرح الأمر: تهيئة الملف الجديد ليكون نظام ملفات تبادل (Swap).

```
swapon /swap_file1
```

شرح الأمر: تفعيل ملف السواب الثاني (`swap_file1/`) لزيادة إجمالي مساحة الـ Swap في النظام.

```
free -m
```

شرح الأمر: الفحص النهائي لحالة الذاكرة وإجمالي مساحة الـ Swap بعد تفعيل الملف الثاني لتظهر قد زادت بمقدار 1 جيجابايت إضافية.

هذه هي الطريقة الثانية لإنشاء وإدارة مساحة التبادل باستخدام ملفات الـ (SWAP File).

Setting the SWAP Space Priority

- By default, the system uses swap spaces in series, meaning that the kernel uses the first activated swap space until it is full, then it starts using the second swap space. However, you can define a priority for each swap space to force that order.
- To set the priority, use the `pri` option in `/etc/fstab`. The kernel uses the swap space with the highest priority first. The default priority is `-2`.
- The following example shows three swap spaces defined in `/etc/fstab`. The kernel uses the last entry first, with `pri=10`. When that space is full, it uses the second entry, with `pri=4`. Finally, it uses the first entry, which has a default priority of `-2`.
- Use `swapon --show` to display the swap space priorities.
- When swap spaces have the same priority, the kernel writes to them in a round-robin

UUID=af30cbb0-3866-466a-825a-58889a49ef33	swap	swap	defaults	0 0
UUID=39e2667a-9458-42fe-9665-c5c854605881	swap	swap	pri=4	0 0
UUID=fbd7fa60-b781-44a8-961b-37ac3ef572bf	swap	swap	pri=10	0 0

إليك الشرح التفصيلي لآلية عمل أولوية مساحات التبادل (SWAP Priority) في لينكس استناداً إلى الصورة المرفقة:

- آلية الاستخدام الافتراضية:
 - افتراضياً، يستخدم النظام مساحات الـ Swap على التوالي (in series)، حيث يبدأ الكيرنل بالكتابة في مساحة الـ Swap الأولى التي يتم تفعيلها حتى تمتلئ تماماً، ثم ينتقل تلقائياً لاستخدام المساحة الثانية وهكذا.
- نظام الأولويات والأرقام (Priority):
 - يمكنك التحكم في ترتيب وس سرعة استخدام مساحات الـ Swap عبر تحديد أولوية (Priority) لكل مساحة داخل ملف `etc/fstab/`.
 - الأولوية الافتراضية: إذا لم تقم بتحديد أولوية يدوياً، فإن أول مساحة سواب يأخذها النظام بـ priority افتراضية تساوي -2 ، وما يليها يتدرج تلقائياً (مثل -3 ثم -4 وهكذا).
 - القاعدة الأساسية للأولوية: الكيرنل يتعامل دائماً مع المساحة ذات الأولوية الأعلى (Highest Priority) أولاً. وكلما كان الرقم موجباً وكبيراً، كلما زادت الأولوية وأصبح هو المساحة المفضلة للكتابة أولاً.

○ تحليل المثال الموضح في الصورة:

○ يوجد 3 مساحات سواب مُعرّفة داخل ملف `etc/fstab/` بترتيب مختلف للأولويات:

1. المساحة الأولى (الأخيرة في الصورة): تحمل قيمة `pri=10` (وهي الأعلى أولوية). لذا يبدأ الكيرنل بالكتابة فيها أولاً حتى تمتلئ.

2. المساحة الثانية: تحمل قيمة `pri=4` ، وعندما تمتلئ المساحة الأولى بالكامل، ينتقل الكيرنل إليها.

3. المساحة الثالثة: تحمل الأولوية الافتراضية `2-` (وهي الأقل)، فلا يتم استخدامها إلا بعد امتلاء المساحتين السابقتين تماماً.

○ تساوي الأولويات (Round-Robin):

○ إذا كانت مساحات الـ Swap تمتلك نفس الأولوية تماماً، فإن الكيرنل لا يملأ واحدة تلو الأخرى، بل يوزع الكتابة عليها بالتساوي بطريقة تُسمى (Round-Robin) لرفع كفاءة الأداء وتوزيع الضغط على الأقراص.

○ أمر العرض والتتبع:

○ لمعرفة الأولويات الحالية وتتبع أي مساحة يتم استخدامها بأي أولوية، تستخدم الأمر:

```
swapon --show
```

```
swapon -s
```

○ **شرح الأمر:** يعرض مساحات الـ Swap المفعلة حالياً في النظام بشكل جدول تفصيلي يوضح اسم الملف أو البارتیشن (Filename)، نوعه (Type)، حجمه بالميجابايت (Size)، المساحة المستهلكة منه (Used)، ودرجة الأولوية لكل مساحة (Priority). ويظهر في النتيجة تدرج الأولويات الافتراضية بدءاً من `2-` وتنازلياً لكل مساحة جديدة يتم تفعيلها تباعاً.

○ **شرح الأمر:** فتح ملف إعدادات النظام `etc/fstab/` لتعديل أو إضافة خصائص مساحات التبادل وتخصيص الأولوية.

○ **السطر الأخير المضاف داخل الملف لتعديل الأولوية:**

```
swap_file1 none swap pri=10 0 0/
```

حيث تم تحديد القيمة المخصصة `pri=10` لرفع أولوية هذا الملف ليكون هو الأعلى أولوية ويتم الاستخدام والكتابة فيه أولاً قبل باقي مساحات السواب.

`swapon -s` (ملاحظة حول تعليق النظام والتحديثات)

○ **شرح الأمر:** ستلاحظ أن النظام علق بسبب التحديثات الجديدة ولم يشعر بالتغييرات المضافة في الملف فوراً، والحل العملي لتجاوز هذا التعليق وقراءة التعديلات هو إيقاف تفعيل جميع مساحات الـ Swap الحالية ثم إعادتها مرة أخرى لكي يقوم الكيرنل بتحديث حالته وتطبيق الأولويات المحدثة.

```
swapoff -av
```

○ **شرح الأمر:** إلغاء تفعيل كافة مساحات الـ Swap المفعلة في النظام دفعة واحدة، لإيقاف العمل بالإعدادات القديمة وإسقاطها من الذاكرة.

```
swapon -av
```

○ **شرح الأمر:** إعادة تفعيل جميع مساحات الـ Swap المسجلة في ملف الـ `etc/fstab/` دفعة واحدة لكي يقرأ النظام التعديلات الجديدة ويطبق الأولويات المحدثة (مثل `pri=10`) ويعتمدها فوراً.

Summary

• In this chapter, you learned:

- You use the **parted** command to add, modify, and remove partitions on disks with the MBR or the GPT partitioning scheme.
- You use the **mkfs.xfs** command to create XFS file systems on disk partitions.
- You need to add file-system mount commands to **/etc/fstab** to make those mounts persistent.
- You use the **mkswap** command to initialize swap spaces.



RH134

Managing Logical Volumes



Red Hat | Enterprise Linux

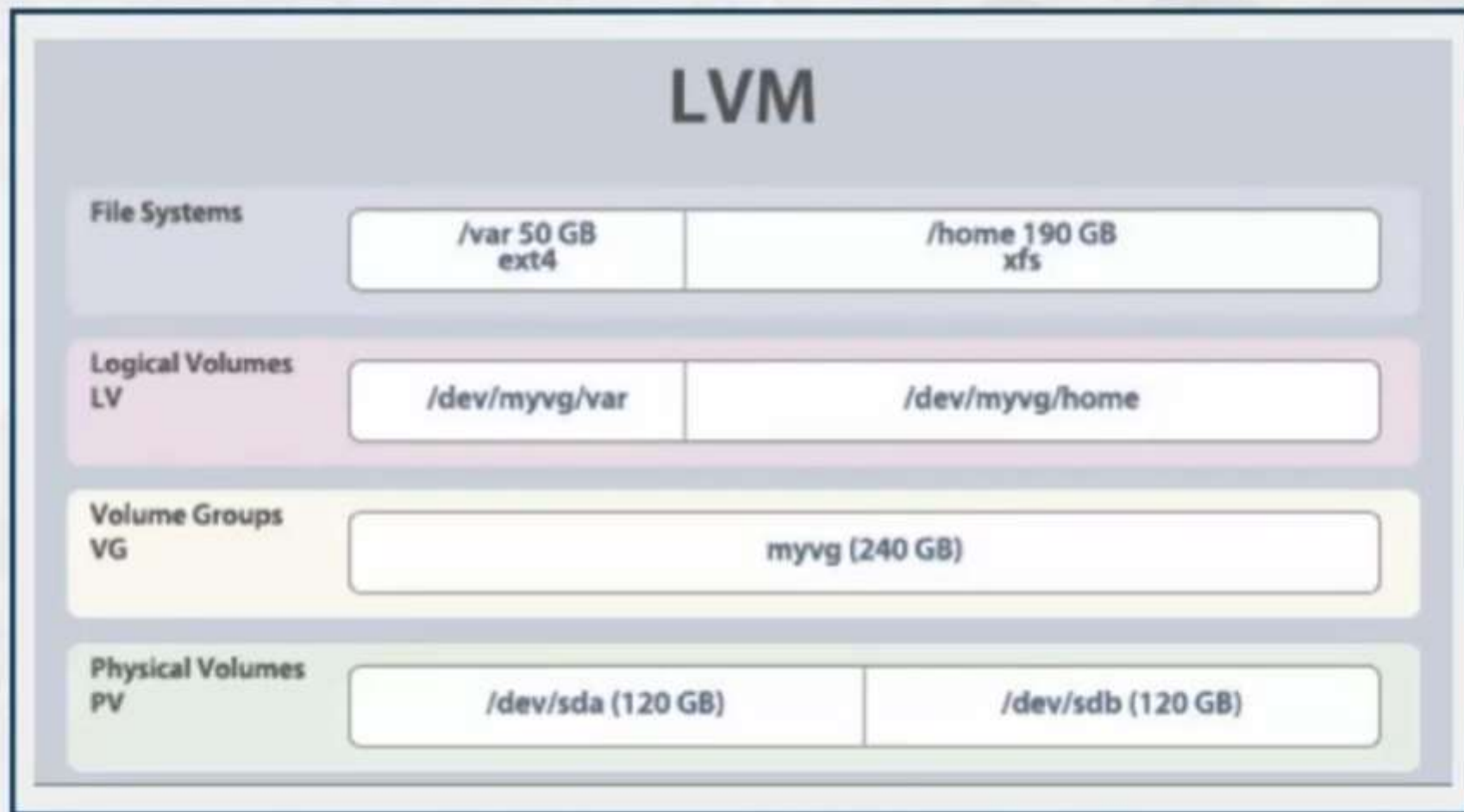
Goal

- Create and manage logical volumes containing file systems and swap spaces from the command line.

Objectives

- Create and manage logical volumes from storage devices, and format them with file systems or prepare them with swap spaces.
- Add and remove storage assigned to volume groups, and nondestructively extend the size of a logical volume formatted with a file system.

Managing Logical Volume Management (LVM) Storage



المشكلة والحل في إدارة التخزين التقليدي مقابل LVM

إذا كان لديك hard disk مساحته 120GB، فلن تستطيع بالطرق التقليدية القديمة أن تقم بإنشاء partition واحد بحجم أكبر من مساحة ال hard disk (مثل طلبك بإنشاء مساحة حجمها 190GB). لحل هذه المشكلة وتجاوز هذا القيد، تأتي تقنية Logical Volume Management (LVM) لتدعم دمج (Combine) أكثر من hard disk معاً ليعملوا ك Single Disk بمجموع المساحة الخاصة بهم، ثم تقطيعهم وإنشاء ال partitions بالأحجام التي تريدها بحرية تامة دون أن يحدث لي أي مشكلة أو كراش في النظام.

مقارنة بين التخزين العادي (Standard Storage) و(LVM):

- Standard Storage (التخزين القياسي): يعتمد على التقسيم المباشر لل hard disk، حيث تكون المساحة مقيدة بحجم ال hard disk الفعلي ولا يمكن تجاوزها أو دمج قرصين معاً بسهولة كمساحة واحدة لتخطي حدود القرص الواحد.
- Logical Volume Management (LVM): يتيح مرونة هائلة في دمج أكثر من hard disk لتكوين pool واحدة وإعادة تقسيمها بأحجام مرنة (Logical Volumes) تتجاوز مساحة القرص الفردي.

شرح طبقات LVM (من البداية للنهاية ومن تحت لفوق حسب الصورة)

تعتمد هيكلية الـ LVM على أربع طبقات أساسية مترابطة يتم شرحها بالتفصيل الممل كالتالي:

1. طبقة Physical Volumes (PV):

- هي الـ **hard disk** الفعلي الذي تقوم بتخزين الداتا عليه بأنواعه المختلفة.
- ممكن بقى يكون **hard disk** بأنواعه بقى يعني ممكن يكون مجموعة من الأقراص أو ما شابه.
- لكي يبدأ النظام في استخدام أي **hard disk** داخل بيئة الـ LVM، يجب أولاً تحضيره وتهيئته عن طريق إنشاء ما يُسمى بـ **Physical Volumes** لهذه الـ **hard disks**.
- وهو كأنك عملت تهيئة للـ **disks** دي عشان تبدأ تستخدمها بعد كده.
- وبتبدأ تعمل حاجة اسمها **volume groups**، وهو ده الـ **pool** بتاعتك اللي انت بتأخذ منها المساحة.
- والـ **volume group** بيكون المجموع الكلي للـ **disks** دي.
- في المثال الموضح بالصورة، يوجد لديك اثنين من الـ **hard disk**: الأول مساحته 120GB ويظهر بالمسار `dev/sda/` ، والثاني مساحته 120GB ويظهر بالمسار `dev/sdb/`.

2. طبقة Volume Groups (VG):

- بعد الانتهاء من تجهيز الـ **Physical Volumes**، تدخل على الطبقة التالية وهي **Volume Groups**.
- هنا تقوم بدمج (Combine) الـ **hard disks** معاً ليتم معاملتهم كـ **Single Disk** كبير واحد يجمع إجمالي مساحاتهم.
- في مثالنا بالصورة، تم تجميع الديسكين معاً في **Volume Group** واحدة أطلق عليها اسم `myvg` وأصبح إجمالي الـ **size** لها هو 240GB (ناتج جمع 120GB + 120GB).

3. طبقة Logical Volumes (LV) وتوضيح مسارات الـ `dev/` :

- بعد تكوين الـ Volume Group الكبيرة، تأتي لخطوة تقطيعها وتقسيمها حسب احتياجك عبر طبقة الـ Logical Volumes.
- هنا تدخل على الليبر اللي هي بتاعت الـ Logical Volumes بتحدد Volume ده بس وبتعمل الـ 190GB جيجابايت لو انت عندك مساحة تسمح بذلك والـ system مش هيكش عليك.
- في الصورة تم تقسيم الـ 240GB إلى جزأين: جزء خاص بـ `home/` ومساحته 190GB وجزء آخر خاص بـ `var/` ومساحته 50GB.
- توضيح النقطة الخاصة بالمسارات (الفكرة ببساطة):
الباث أو المسار بتاع الـ logical volume بيكون موجود دائماً تحت مجلد الأجهزة الأساسي `dev/` . والموضوع يتم بالترتيب التالي:

a. بمجرد أن تنشئ الـ volume group (وسمينها مثلاً `myvg`)، يقوم النظام تلقائياً بإنشاء مجلد (Folder) داخل `dev/` يحمل اسم الـ volume group الخاصة بك ليكون هكذا: `dev/myvg/`.

b. بعد ذلك، أول ما تقم بإنشاء الـ logical volumes (مثل `home` و `var`)، فإن النظام يضع هذه الـ logical volumes كملفات أجهزة بداخل ذلك المجلد الذي تم إنشاؤه للـ volume group، فتصبح المسارات النهائية للوصول إليها هي: `dev/myvg/home/` و `dev/myvg/var/`.

- بعد أن قمت بإنشاء ال Logical Volumes الخاصة بك، ستجدها ظهرت في هذه الطبقة الأخيرة
- وهذا ال File System هو اللي أنت بتستخدمه عشان ترن ال system بتاعك.
- هذه هي الطبقة التي يتم فيها وضع ال File System المناسب لكل partition لكي تستطيع استخدامها في تخزين الملفات وقراءتها.
- في المثال الموضح: جزء ال `var/` تم تهيئته بـ File System نوعه `ext4` ومساحته 50GB، وجزء ال `home/` تم تهيئته بـ File System نوعه `xfs` ومساحته 190GB.
- هذا هو ال File System النهائي الذي تستخدمه فعلياً لإدارة وتشغيل نظامك وداتا الخاصة بك بكفاءة تامة.

ملحوظة هامة بعد إنشاء ال Logical Volumes:

بعد الانتهاء من إنشاء ال Logical Volumes، قم بعمل تهيئة لها بـ File System (مثل `mkfs`)، ثم اربطها بـ Mount Point مناسب، ولا تنسَ في النهاية أن تحدث إعداداتها في ملف `etc/fstab/` لضمان تثبيتها واستمرارها تلقائياً مع كل عملية إقلاع للنظام.

Logical Volume Managment (LVM) Concepts

Logical volumes and logical volume management make it easier to manage disk space. If a file system that hosts a logical volume needs more space, it can be allocated to its logical volume from the free space in its volume group and the file system can be resized. If a disk starts to fail, a replacement disk can be registered as a physical volume with the volume group and the logical volume's extents can be migrated to the new disk.

LVM Definitions

➤ Physical devices

Physical devices are the storage devices used to save data stored in a logical volume. These are block devices and could be disk partitions, whole disks, RAID arrays, or SAN disks. A device must be initialized as an LVM physical volume in order to be used with LVM. The entire device will be used as a physical volume.

مفاهيم إدارة الأقراص المنطقية (Logical Volume Management- LVM Concepts)

- تقنيات إدارة الأقراص المنطقية (Logical volumes و Logical Volume Management) بتسهل جداً إدارة مساحة التخزين (Disk space).
- لو ال File system اللي شايل ال Logical volume محتاج مساحة زيادة، يقدر النظام يخصصها لل Logical volume مباشرة من ال Free space (المساحة الفارغة) الموجودة جوه ال Volume group، ويقدر يعمل توسيع وتكبير لل File system بكل سهولة.
- لو فيه Disk بدأ يحصل فيه عطل أو يبيوط (Starts to fail)، يقدر النظام يسجل Disk بديل ك Physical volume جوه ال Volume group ويتم ترحيل (Migrated) ال Extents الخاصة بال Logical volume لل Disk الجديد بأمان تام.

تعريفات ال (LVM Definitions)

Physical devices (الأجهزة الفيزيائية)

- **التعريف الأساسي:** هي وحدات التخزين الفعلية (Storage devices) اللي بنستخدمها عشان نحفظ الداتا اللي متخزنة جوه ال Logical volume.
- **أنواع الأجهزة:** تعتبر هذه الأجهزة عبارة عن block devices، ويمكن أن تكون أشكالها متنوعة مثل Disk partitions، أو Whole disks، أو RAID arrays، أو SAN disks.
- **شروط الاستخدام:** عشان أي جهاز أو Disk يقدر يتفاعل ويشغل داخل بيئة ال LVM، لازم يتم تهيئته وتجهيزه أولاً ك LVM physical volume.
- **نطاق الاستخدام:** يتم استخدام The entire device أو ال partition بالكامل ك Physical volume.

LVM Definitions

➤ Physical volumes (PVs)

You must initialize a device as a physical volume before using it in an LVM system. LVM tools segment physical volumes into physical extents (PEs), which are small chunks of data that act as the smallest storage block on a physical volume.

➤ Volume groups (VGs)

Volume groups are storage pools made up of one or more physical volumes. This is the functional equivalent of a whole disk in basic storage. A PV can only be allocated to a single VG. A VG can consist of unused space and any number of logical volumes.

➤ Logical volumes (LVs)

Logical volumes are created from free physical extents in a volume group and provide the "storage" device used by applications, users, and the operating system. LVs are a collection of logical extents (LEs), which map to physical extents, the smallest storage chunk of a PV. By default, each LE maps to one PE. Setting specific LV options changes this mapping; for example, mirroring causes each LE to map to two PEs.

Physical volumes (PVs):

- يجب عليك تهيئة الجهاز أو الـ device كـ physical volume قبل استخدامه في نظام الـ LVM.
- أدوات الـ LVM تقسم الـ physical volumes إلى physical extents (PEs)، وهي عبارة عن أجزاء صغيرة من البيانات تعمل كأصغر وحدة تخزين (storage block) داخل الـ physical volume.

Volume groups (VGs):

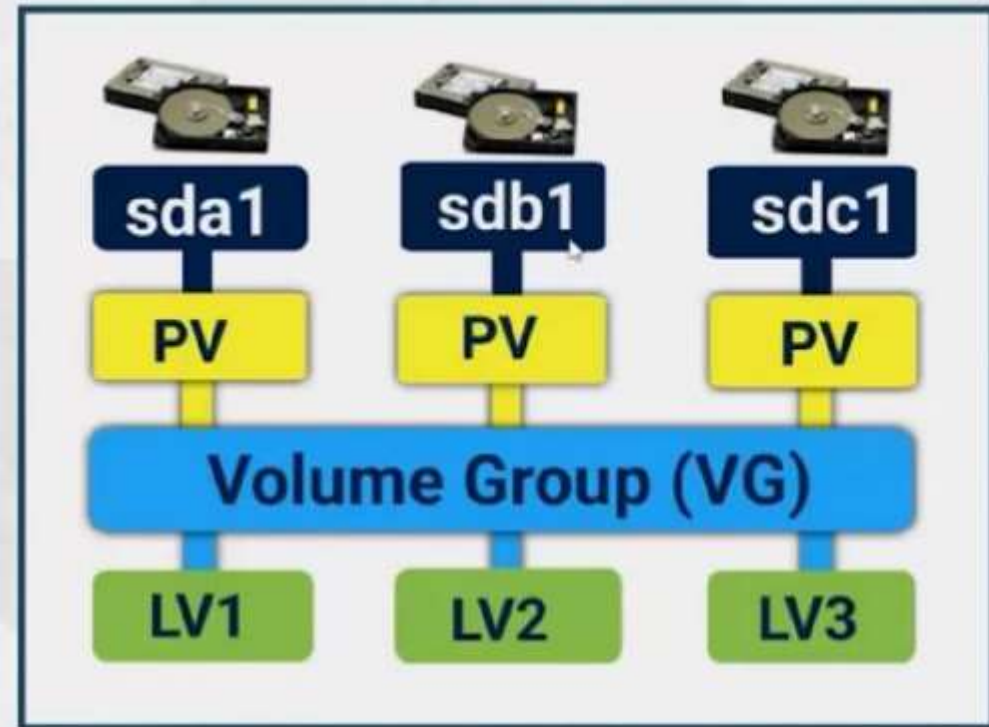
- الـ Volume groups هي عبارة عن مساحات تخزين تجميعية (storage pools) تتكون من واحد أو أكثر من الـ physical volumes.
- تُعتبر هذه الطريقة مكافئة وظيفياً لقرص كامل (whole disk) في التخزين الأساسي (basic storage).
- الـ PV الواحد يمكن تخصيصه لـ VG واحد فقط (single VG).
- الـ VG يمكن أن يتكون من مساحة فارغة (unused space) وأي عدد من الـ logical volumes.

Logical volumes (LVs):

- يتم إنشاء الـ Logical volumes من الـ free physical extents داخل الـ volume group، وتوفر جهاز "التخزين" (storage device) الذي تستخدمه التطبيقات والمستخدمون ونظام التشغيل.
- الـ LVs هي مجموعة من الـ logical extents (LEs) التي ترتبط وتتطابق مع الـ physical extents، وهي أصغر وحدة تخزين للـ PV.
- افتراضياً (By default)، كل LE يرتبط بـ PE واحد.
- ضبط خيارات محددة للـ LV يغير هذا الارتباط؛ على سبيل المثال، التكرار أو النسخ المتطابق (mirroring) يتسبب في ارتباط كل LE باثنين من الـ PEs.

Why Logical volume management (LVM) ?

- Volumes can consist of more than one disk.
- Easy resize operation.
- Easy replacement of failing disks.
- Advanced options such a working with snapshots.
- Easy to add new volumes.
- Easy to add many volumes.
- Up to 256 logical volume.



لماذا نستخدم إدارة الأقراص المنطقية (LVM - Why Logical Volume Management)؟

- **Volumes can consist of more than one disk**: ال Volumes أو ال Storage units لا تكون مقيدة بـ Hard disk واحد، بل يمكنك دمج أكثر من Disk معاً لتعمل كوحدة تخزين واحدة (Single Disk) بمساحة إجمالية ضخمة.
- **Easy resize operation**: سهولة إجراء عمليات ال Resize وتعديل حجم ال Logical volumes أو ال File systems في أي وقت، سواء بالتوسيع أو التكبير دون الحاجة لحذف ال Data أو إعادة التقسيم من جديد.
- **Easy replacement of failing disks**: سهولة استبدال ال Disks التي تعطلت أو بدأت في التلف (Failing disks) عن طريق تسجيل Hard disk جديد وترحيل ال Extents إليه بأمان تام ودون توقف النظام.
- **Advanced options such as working with snapshots**: توفر خيارات متقدمة جداً مثل استخدام ال Snapshots (أخذ صورة لحظية للحالة)، والتي تتيح لك توثيق حالة ال Storage volumes في وقت معين للرجوع إليها عند إجراء تعديلات خطيرة.
- **Easy to add new volumes**: سهولة إضافة New volumes أو وحدات تخزين جديدة بكل مرونة ودون أي تعقيد في البنية الأساسية.
- **Easy to add many volumes**: القدرة السلسة على إضافة وإدارة العديد من ال Volumes وال Storage units الكثيرة داخل النظام حسب متطلبات التطبيقات والمستخدمين.
- **Up to 256 logical volumes**: يدعم النظام إنشاء ما يصل إلى 256 Logical volumes كحد أقصى داخل نطاق الإدارة، مما يمنحك حرية تكامل هائلة لتنظيم وتوزيع مساحات ال Storage.

ميزة أخرى للـ LVM وهي إنها بتدعم الـ VDO.

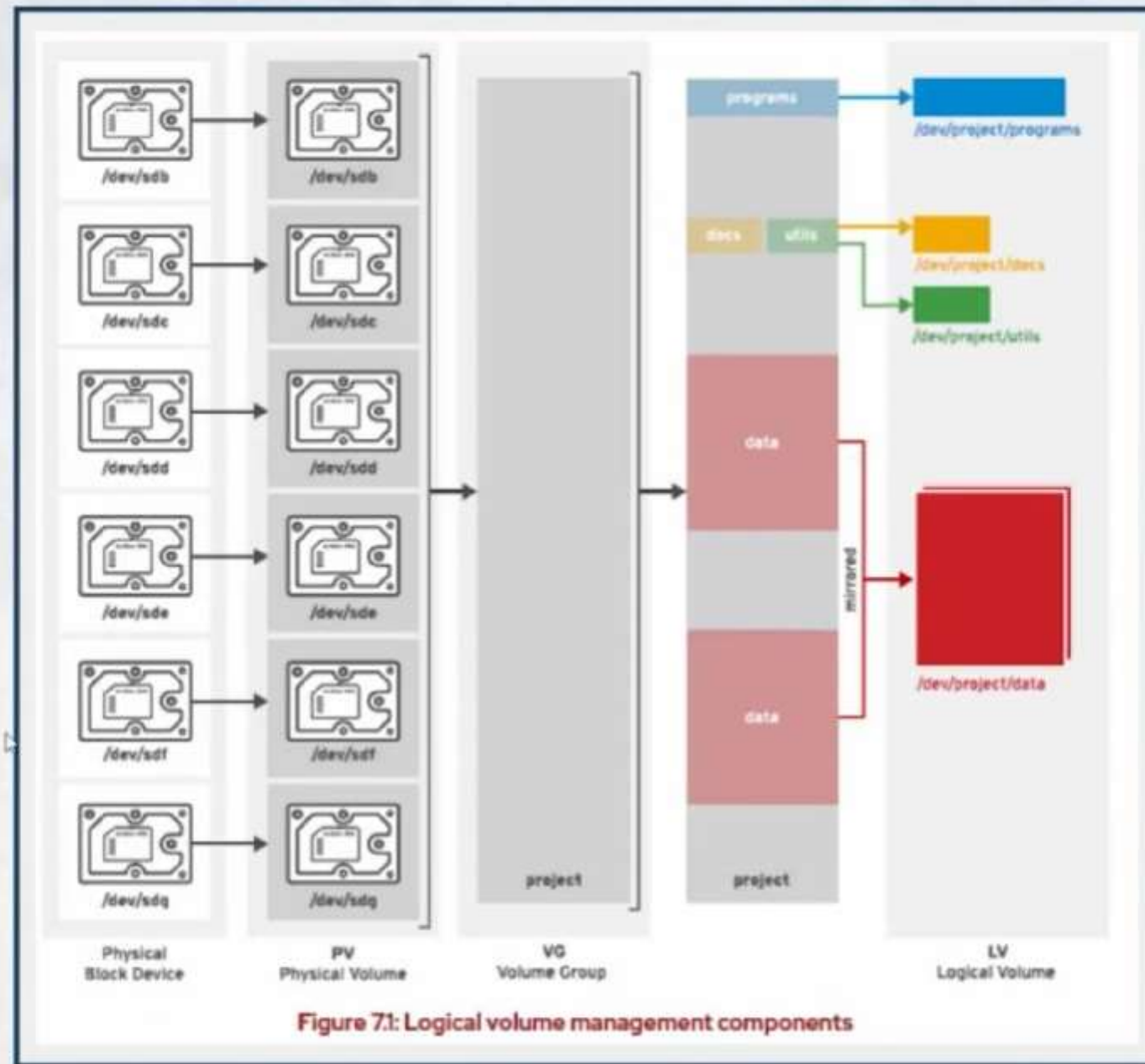
VDO (Virtual Data Optimizer) هي خدمة متقدمة جداً يتم دمجها مع الـ LVM في نظام الـ RHEL، وبتلعب دوراً أساسياً في توفير مساحات ضخمة جداً من الـ Storage من خلال تقليل حجم البيانات المخزنة (Data Reduction).

تعتمد الخدمة بشكل أساسي على تقنيتين رئيسيتين:

- **Deduplication (إزالة التكرار):** إذا كان لديك بيانات أو ملفات متطابقة مكررة في أكثر من مكان داخل الـ Storage (مثل عدة مستخدمين قاموا بتنزيل نفس الملفات أو التطبيقات)، يقوم الـ VDO باكتشاف هذه القطع المتطابقة وتخزين نسخة واحدة فقط منها، مع جعل باقي الملفات تشير إليها بدلاً من إعادة تخزينها مراراً وتكراراً، مما يوفر مساحة هائلة.
- **Compression (الضغط):** يقوم بضغط كتل البيانات (Data blocks) قبل كتابتها فعلياً على الـ Hard disk، بحيث تأخذ مساحة فيزيائية أصغر بكثير من حجمها الأصلي، تماماً مثل ضغط الملفات ولكنها تعمل تلقائياً وخلف الكواليس على مستوى الـ Block device.

كيف تستفيد منها مع الـ LVM؟

عند ربط الـ VDO مع الـ Logical Volumes، يتيح لك النظام الحصول على مساحة تخزينية ظاهرية (Logical size) أكبر بكثير من المساحة الفيزيائية الحقيقية للـ Hard disks المتاحة لديك، مما يرفع كفاءة استغلال التخزين إلى أقصى حد ممكن.



Create a physical Volume

- Use **pvcreate** to label the partition (or other physical device) as a physical volume. The **pvcreate** command divides the physical volume into physical extents (PEs) of a fixed size, for example, 4 MiB blocks. You can label multiple devices at the same time by using space-delimited device names as arguments to **pvcreate**.

```
[root@host ~]# pvcreate /dev/vdb2 /dev/vdb1
```

Create a physical Volume (إنشاء Physical Volume):

- استخدام أمر الـ **pvcreate** لعمل label (تسمية) للـ **partition** أو أي **physical device** آخر لكي يتم تعريفه كـ **physical volume**.
- أمر الـ **pvcreate** يقوم بتقسيم الـ **physical volume** إلى وحدات تُسمى **physical extents (PEs)** ذات حجم ثابت (**fixed size**)، مثل كتل بحجم 4 MiB.
- يمكنك عمل label لأكثر من **device** في نفس الوقت عن طريق كتابة أسماء الـ **devices** مفصولة بمسافات (**space-delimited device names**) كـ **arguments** لأمر الـ **pvcreate**.

Create a Volume Group

- Use **vgcreate** to collect one or more physical volumes into a volume group. A volume group is the functional equivalent of a hard disk; you will create logical volumes from the pool of free physical extents in the volume group.
- The **vgcreate** command-line consists of a volume group name followed by one or more physical volumes to allocate to this volume group.

```
[root@host ~]# vgcreate vg01 /dev/vdb2 /dev/vdb1
```

- This creates a VG called **vg01** that is the combined size, in PE units, of the two PVs **/dev/vdb2** and **/dev/vdb1**.
- A VG only needs to be created if none already exist. Additional VGs may be created for administrative reasons to manage the use of PVs and LVs. Otherwise, existing VGs can be extended to accommodate new LVs when needed.

Create a Volume Group (إنشاء Volume Group):

- استخدام أمر الـ `vgcreate` لجمع واحد أو أكثر من الـ `physical volumes` معاً لتشكيل `volume group`. الـ `Volume group` يمثل المكافئ الوظيفي لـ `hard disk`؛ ومن خلاله ستتمكن من إنشاء `logical volumes` من مساحة الـ `pool` الخاصة بـ الـ `free physical extents` داخل الـ `volume group`.
- يتكون سطر الأوامر لأمر الـ `vgcreate` من اسم الـ `volume group` متبوعاً بـ `physical volume` واحد أو أكثر يتم تخصيصهم لهذا الـ `volume group`.

مثال تطبيقي على الـ Terminal:



Bash

```
[root@host ~]# vgcreate vg01 /dev/vdb2 /dev/vdb1
```

- هذا الأمر ينشئ `VG` يسمى `vg01` بمساحة إجمالية مجتمعة (محسوبة بوحدات الـ `PE units`) للـ `PVs` وهما `dev/vdb1/` و `dev/vdb2/`.
- الـ `VG` يلزم إنشاؤه فقط إذا لم يكن هناك أي `VG` موجود مسبقاً. يمكن إنشاء `additional VGs` لأسباب إدارية لإدارة استخدام الـ `PVs` و الـ `LVs`، وإلا فيمكن توسيع الـ `existing VGs` الحالية لاستيعاب `new LVs` عند الحاجة.

لماذا قد يحتاج الـ System Administrator إلى أكثر من (VG) Volume Group منفصل بدلاً من الاكتفاء بـ Volume Group واحد وتوسيعه دائماً؟

○ فصل الأداء حسب نوع الأقراص (Performance & Hardware Separation):

○ **الموقف:** عندما يحتوي السيرفر على أنواع مختلفة من وحدات التخزين الفيزيائية، مثل أقراص SSD سريعة جداً وأقراص HDD تقليدية بطيئة ولكنها توفر مساحات ضخمة.

○ **لماذا لا نكتفي بـ VG واحد؟** لو قمنا بدمج جميع الأقراص المختلفة في Volume Group واحد ضخم، فإن نظام الـ LVM سيقوم بتوزيع الـ Physical Extents (PEs) عشوائياً، ولن نستطيع ضمان أن الـ Logical Volume (LV) الخاص بقاعدة البيانات الحساسة (Database) سيتم إنشاؤه على أقراص الـ SSD السريعة.

○ **الحل الإداري:** يقوم الـ System Administrator بإنشاء Volume Group مستقل للأقراص السريعة (مثلاً `vg_fast`) يخصص له أقراص الـ SSD فقط لإنشاء الـ LVs الهامة عليها، و Volume Group آخر للأقراص العادية (`vg_slow`) للملفات التي لا تتطلب أداءً فائقاً.

○ عزل الموارد والأقسام (Fault & Department Isolation):

○ **الموقف:** عندما يكون لديك أكثر من Department (قسم) أو بيئات عمل مختلفة (مثل بيئة التطوير Development البيئة الحية Production)، وكل قسم يمتلك مجموعة من الأقراص الخاصة به (مثلاً 3 أقسام، وكل قسم لديه 3 أقراص بإجمالي 9 أقراص).

○ **لماذا لا ندمج الكل في VG واحد؟** إذا قمنا بجمع التسعة أقراص معاً في Volume Group واحد، فسيكون التخزين مشتركاً وغير معزول؛ مما يعني أن امتلاء مساحة التخزين الخاصة بقسم معين أو إساءة استهلاك الموارد قد يؤثر سلباً أو يتداخل مع استقرار الأقسام الأخرى.

○ **الحل الإداري:** فصل كل قسم في Volume Group مستقل بذاته (حيث ينضم كل 3 أقراص تخصص قسمياً معيناً في VG خاص بهم)، مما يمنع تداخل الاستهلاك، ويضمن عدم تأثر باقي الأقسام في حال حدوث امتلاء أو مشكلة في مساحة قسم آخر، و يمنحك تحكماً إدارياً كاملاً في توزيع موارد الـ Storage.

Creating a Logical Volume

- Use **lvcreate** to create a new logical volume from the available physical extents in a volume group. At a minimum, the **lvcreate** command includes the **-n** option to set the LV name, either the **-L** option to set the LV size in bytes or the **-l** option to set the LV size in extents, and the name of the volume group hosting this logical volume.

```
[root@host ~]# lvcreate -n lv01 -L 700M vg01
```

- This creates an LV called lv01, 700 MiB in size, in the VG vg01. This command will fail if the volume group does not have a sufficient number of free physical extents for the requested size.
- Note also that the size will be rounded to a factor of the physical extent size if the size cannot match exactly.
- The following list provides some examples of creating LVs:
 - **lvcreate -L 128M**: Size the logical volume to exactly 128 MiB.
 - **lvcreate -l 128**: Size the logical volume to exactly 128 extents.

Creating a Logical Volume (إنشاء Logical Volume):

- استخدام أمر الـ `lvcreate` لإنشاء Logical Volume جديد من الـ `available physical extents` المتاحة داخل الـ `Volume Group`.
- يتضمن الأمر في حده الأدنى الخيارات الآتية:
 - خيار الـ `-n` : لتحديد اسم الـ Logical Volume (مثل `lv01`).
 - اسم الـ `Volume Group` المستضيف في نهاية الأمر (مثل `vg01`).
 - الفرق بين خيار تحديد الحجم (`-L` و `-l`):
 - خيار الـ `-L` (Uppercase / الحرف الكبير): تستخدمه لتحديد حجم الـ Logical Volume مباشرة بالميغابايت أو الغيغابايت (مثل `-L 700M` أو `-L 128M`), ويجب عليك التأكد من أن هذه المساحة متوفرة كـ `free space` في الـ `Volume Group`.
 - خيار الـ `-l` (Lowercase / الحرف الصغير): تستخدمه لتحديد حجم الـ Logical Volume بناءً على عدد الـ `extents` (مثل `-l 128`). فإذا أعطيته الرقم 128, فإن الحجم يُحسب بعدد الـ `extents`, وبما أن الـ `extent` الواحد يساوي `4 MiB` افتراضياً, فعند ضرب `128 × 4 MiB` سيساوي الحجم الإجمالي `512 MiB`.

أمثلة توضيحية من الـ Slide:

- `lvcreate -n lv01 -L 700M vg01`: ينشئ LV باسم `lv01` وبحجم `700 MiB` داخل الـ `VG` المسمى `vg01`. (ملاحظة: سيفشل الأمر إذا لم تكن الـ `Volume Group` تمتلك عدداً كافياً من الـ `free physical extents` بالحجم المطلوب).
- `lvcreate -L 128M`: يحدد حجم الـ Logical Volume ليكون `128 MiB` بالضبط.
- `lvcreate -l 128`: يحدد حجم الـ Logical Volume ليكون `128 extents` بالضبط.
- **ملاحظة هامة:** سيقوم النظام بتقريب الحجم (`rounded`) تلقائياً ليطابق مضاعفات حجم الـ `physical extent size` إذا لم يكن الحجم مطابقاً تماماً.

Add the file system

- Use **mkfs** to create an **XFS** file system on the new logical volume. Alternatively, create a file system based on your preferred file system, for example, **ext4**.

```
[root@host ~]# mkfs -t xfs /dev/vg01/lv01
```

- To make the file system available across reboots, perform the following steps:

```
[root@host ~]# mkdir /mnt/data
```

- Add an entry to the `/etc/fstab` file:

```
/dev/vg01/lv01 /mnt/data xfs defaults 1 2
```

- Run **mount /mnt/data** to mount the file system that you just added in `/etc/fstab`.

```
[root@host ~]# mount /mnt/data
```

Removing a Logical Volume

- To remove all logical volume components, perform the following steps:

- **Prepare the file system.**

```
[root@host ~]# umount /mnt/data
```

- **Remove the logical volume.**

```
[root@host ~]# lvremove /dev/vg01/lv01
```

- **Remove the volume group.**

```
[root@host ~]# vgremove vg01
```

- **Remove the physical volumes.**

```
[root@host ~]# pvremove /dev/vdb2 /dev/vdb1
```

Top-to-> LV -> Mount VG -> (>), عند الرغبة في الحذف أو التفكيك، يجب أن تبدأ من الأعلى إلى الأسفل (Bottom):

- 1. Prepare the file system (`umount /mnt/data` - فك التوصيل):
 - هو إجراء آمن ومؤقت؛ حيث يقوم بقطع اتصال ال **File system** بنظام التشغيل فقط.
 - إذا قمت بعمل **mount** مرة أخرى بعد هذه الخطوة، ستجد نفس الداتا موجودة تماماً ولم يتم حذف أي شيء.
- 2. Remove the logical volume (`lvremove /dev/vg01/lv01` - حذف ال LV):
 - عند تنفيذ هذا الأمر، فإنك تقوم بحذف ال **Logical Volume** بالكامل مع كل الداتا والملفات الموجودة عليه، تماماً كأنك قمت بحذف **Partition** بالكامل بداخله محتوياته، وتصبح المساحة غير قابلة للاسترجاع بسهولة.
- 3. Remove the volume group (`vgremove vg01` - حذف ال VG):
 - بعد حذف ال LVs، تأتي خطوة حذف ال **Volume Group** نفسه؛ وبذلك تلغي الحاوية الكبرى التي كانت تجمع الأقراص ببعضها، وتحرر ال **Physical Volumes** وتنفصل عن بعضها البعض.
- 4. Remove the physical volumes (`pvremove /dev/vdb2 /dev/vdb1` - إزالة ال PVs):
 - الخطوة الأخيرة هي إزالة خصائص وعلامات ال **LVM** من على الأقراص الفيزيائية.

الخلاصة كما ذكرت تماماً:

لو كنت دمجت ثلاثة ديسكات معاً ووضعت عليها بيانات ثم قررت التراجع وعملت هذه الخطوات مرتبة للنهاية، فإن كل الداتا تطير وتتلاشى بالكامل، وكل ديسك يعود لحالته الأصلية الخام (**Raw disk**) كأن شيئاً لم يكن وتعود أصل الأقراص منفصلة كما كانت.

1. إنشاء وعرض الـ Physical Volumes (PVs):

○ أمر الإنشاء: `pvcreate /dev/nvme0n2 /dev/nvme0n3` لتجهيز الأقراص ووسمها كـ Physical Volumes.

○ أمر العرض السريع (`pvs`): يعرض ملخصاً للـ Physical Volumes، بما في ذلك اسم الـ PV، الـ Volume Group التابع له، والـ Total Size والـ Free Size.

○ أمر العرض التفصيلي (`pvdisk`): يعرض تفاصيل عميقة تشمل الـ Physical Volume Name، الـ Size، الـ UUID المعين له، والـ VG Name (اسم الـ Volume Group).

○ ملحوظة تيسيرية: أثناء كتابة أمر الـ `vgcreate`، يمكنك الضغط على زر (Tab) ليكمل لك أسماء الـ Physical Volumes تلقائياً.

2. إنشاء وعرض الـ Volume Group (VG):

○ أمر الإنشاء: `vgcreate vg01 /dev/nvme0n2 /dev/nvme0n3` لتجميع الـ Physical Volumes معاً داخل حاوية واحدة تُسمى `vg01`.

○ أمر العرض السريع (`vgs`): يعرض ملخصاً ومعلومات الـ Volume Groups الموجودة.

○ مكونات وعرض أمر `vgs` بالتفصيل: يعرض معلومات مثل اسم الـ Volume Group (`VG`)، عدد الـ Physical Volumes المدمجة (`PV#`)، عدد الـ Logical Volumes المنشأة (`LV#`)، عدد الـ Snapshots (`SN#`)، خصائص الـ VG (`Attr`)، المساحة الإجمالية (`VSize`)، والمساحة الخالية المتاحة (`VFree`).

3. إنشاء وعرض ال Logical Volume (LV):

- **أمر الإنشاء:** `lvcreate -n lv01 -L 5G vg01` لإنشاء Logical Volume جديد باسم `lv01` وبحجم `5G` داخل ال Volume Group المسمى `vg01`.
- **أمر العرض السريع (`lvs`):** يعرض ملخصاً لل Logical Volumes.
- **مكونات وعرض أمر `lvs` بالتفصيل:** يعرض اسم ال Logical Volume (`LV`)، اسم ال Volume Group التابع له (`VG`)، الخصائص (`Attr`)، مساحة ال Logical Volume (`LSize`)، بالإضافة إلى تفاصيل أخرى مثل ال Pool و Origin و %Data وغيرها.
- **أمر العرض التفصيلي (`lvdisplay`):** يعرض مسار ال (Path) LV الذي يتم استخدامه لاحقاً في عمليات ال Mount وإنشاء ال File System، مع توضيح ارتباطه بال Volume Group.

4. فحص تفاصيل ال Volume Group وسؤال الإنترفيو (`vgdisplay`):

- **أمر العرض:** `vgdisplay` يعرض خصائص وتفاصيل شاملة لل Volume Group.
- **حجم ال PE:** يوضح خانة ال (Physical Extent Size) PE Size والتي تكون افتراضياً `4 MiB`.
- **سؤال إنترفيو:** إذا طلب منك في مقابلة عمل إنشاء Volume Group بحيث يكون ال Physical Extent الخاص بها بحجم `8 MiB`، يتم استخدام خيار تحديد الحجم `-s` مع أمر الإنشاء (مثل: `vgcreate -s 8M vg01 /dev/nvme0n2`).

```
mkfs -t xfs /dev/vg01/lv01
```

○ استخدام أمر الـ `mkfs` مع خيار الـ `-t xfs` لتهيئة الـ **Logical Volume** وإنشاء نظام ملفات من نوع **XFS** عليه ليصبح مهيئاً لتخزين الملفات والبيانات.

```
mkdir /data_lv01
```

```
mount /dev/vg01/lv01 /data_lv01
```

```
df -h
```

○ استخدام أمر الـ `df -h` (Disk Free) لعرض مساحات الـ **File systems** النشطة والموصولة حالياً، للتأكد من ظهور الـ **Logical Volume** (تحت المسار `dev/mapper/vg01-lv01/`) بحجمه الإجمالي `5.0G` وموصولاً على المسار `data_lv01/`.

```
vi /etc/fstab
```

○ فتح ملف التكوين الرئيسي باستخدام محرر النصوص `vi /etc/fstab` لإضافة سطر التوصيل الثابت، لضمان أن النظام سيعيد عمل **Mount** تلقائياً لهذا الـ **Logical Volume** في كل مرة يتم فيها إعادة تشغيل السيرفر.

○ السطر المضاف داخل الملف: `dev/vg01/lv01 /data_lv01 xfs defaults 0 0/`

```
umount /data_lv01
```

`df -h`

- تنفيذه في البداية قبل الـ `mount` لكي تلاحظ بنفسك أن الـ `Logical Volume` غير ظاهر أو غير موصول بعد في قائمة الـ `Filesystems` النشطة.

`mount -a`

- توضيح بخصوص ظهور الـ `Logical Volume` تلقائياً: أمر الـ `df -h` الأول لم يكن يظهر الـ `LV` لأنه كان في حالة الـ `umount` (مفصولاً). ولكن بمجرد كتابة وتنفيذ أمر الـ `mount -a` , فإن النظام يقرأ ملف التكوين (`etc/fstab/`), ويبحث عن أي مسارات مسجلة فيه ولم تقم بفعل التوصليل لها, فيقوم بتنفيذ الـ `mount` لها تلقائياً. لذلك عندما كتبت `df -h` بعدها, ظهر الـ `LV` موصولاً ومتاحاً دون أن تحتاج لكتابة أمر الـ `mount` اليدوي الطويل.

`vi /etc/fstab`

- تم إزالة الـ `record` الخاص بالـ `Logical Volume` من داخل ملف التكوين لتنظيف النظام قبل البدء في عملية الحذف والتفكيك.

`lvremove /dev/vg01/lv01`

- محاولة الحذف والفشل المؤقت: عند محاولة تنفيذ أمر الحذف مباشرة ظهرت رسالة خطأ تفيد بأن الـ `Logical Volume` لا يزال مستخدماً (`contains a filesystem in use / mounted`).
 - الحل: تم تنفيذ أمر الفك `umount /data_lv01` أولاً لتحريره, وبعدها نجح أمر الحذف تماماً.
- أوامر الحذف والتفكيك النهائية بالتسلسل:

`lvremove /dev/vg01/lv01` ○

`vgremove vg01` ○

`pvremove /dev/nvme0n2 /dev/nvme0n3` ○

Summary of LVM Commands

- Initialize physical volumes.

```
# pvcreate /dev/sdb1 /dev/sdc1 /dev/sdd1  
# pvs  
# pvdisplay
```

- Create volume group "data-vg" with physical volumes.

```
# vgcreate data-vg /dev/sdb1 /dev/sdc1 /dev/sdd1  
# vgs  
# vgdisplay
```

- Create logical volume "data-lv" in volume group.

```
# lvcreate -L 10G -n data-lv data-vg  
# lvs  
# lvdisplay
```

- Use /dev/data-vg/data-lv as a block device.

Summary of LVM Commands

- Removing a logical volume will destroy any data stored on the logical volume.

```
# lvremove /dev/data-vg/data-lv (file system must be unmounted first)
# vgremove data-vg
# pvremove /dev/sdb1 /dev/sdc1 /dev/sdd1
```

- Extending Logical Volumes (no down time):

```
# pvcreate /dev/sde1
# vgextend data-vg /dev/sde1   في حالة لم يكن موجود أصلا free sapce
# lvextend -L +3G /dev/data-vg/data-lv
# xfs_growfs /dev/data-vg/data-lv (update the file system for XFS file systems)

# resize2fs /dev/data-vg/data-lv (update the file system for other file systems)
```

Or:

```
# lvextend -r -L +3G /dev/data-vg/data-lv (extend and update in one step)
```

:Extending Logical Volumes (No Downtime)

- عملية توسيع ال Logical Volume تتم بشكل عام بدون أي توقف (No downtime), مما يعني أنك لست بحاجة لعمل `umount` لل Logical Volume أثناء التوسيع, ويمكن أن يظل ال LV يعمل وموصولاً (`mounted` أو `up and running`) أثناء العملية بالنسبة للأنواع العادية.
- ملحوظة هامة حول استخدام علامة البلس (+):
 - عند كتابة الحجم بدون علامة + (مثل `3G-L`), فإن النظام يقوم بتغيير حجم ال Partition ليصبح الحجم الجديد هو 3 جيجا بالتحديد بغض النظر عن المساحة السابقة.
 - أما عند استخدام علامة البلس (مثل `3G+L`), فإن النظام يقوم بإضافة 3 جيجا جديدة فوق المساحة الحالية الموجودة بالفعل.
- خطوات التوسيع في حال توفر مساحة داخل ال Volume Group (الطريقة الأولى):
 - إذا كان لديك مساحة خالية بالفعل داخل ال Volume Group, فإنك لا تحتاج لإنشاء PV جديد, وتكتفي بتنفيذ خطوات التوسيع مباشرة على ال Logical Volume وتحديث نظام الملفات.
 - خطوات التوسيع في حال عدم توفر مساحة كافية وإضافة هارد جديد (الطريقة الثانية):
 - 1. إنشاء Physical Volume جديد: تجهيز القرص أو البارتيشن الجديد وإدخاله في منظومة ال LVM عبر أمر ال PV:

```
pvcreeate /dev/sde1
```

○ 2. توسيع ال **Volume Group**: إضافة ال **PV** الجديد إلى ال **Volume Group** الحالية لتوسيع المساحة الإجمالية للوعاء التخزيني:

○ `vgextend data-vg /dev/sde1`

○ 3. توسيع ال **Logical Volume**: زيادة مساحة ال **LV** باستخدام أمر التوسيع:

○ `lvextend -L +3G /dev/data-vg/data-lv`

○ 4. تحديث ال **File System** (جعل نظام التشغيل يرى المساحة الجديدة): بعد توسيع مساحة ال **LV** الفيزيائية، يجب تحديث نظام الملفات حتى يتعرف على المساحة المضافة، وذلك أوامر بحسب نوع ال **File System**:

○ الأمر الأول (إذا كان نظام الملفات من نوع **XFS** وهو الافتراضي في ريدهات):

○ `xfs_growfs /dev/data-vg/data-lv`

○ الأمر الثاني (إذا كان نظام الملفات من أنواع أخرى مثل **ext4**):

○ `resize2fs /dev/data-vg/data-lv`

الاختصار في خطوة واحدة (Extend and Update):

○ يمكنك اختصار خطوة توسيع ال **Logical Volume** مع تحديث ال **File System** معاً في سطر واحد باستخدام مفتاح ال **-r** مع أمر ال `lvextend`:

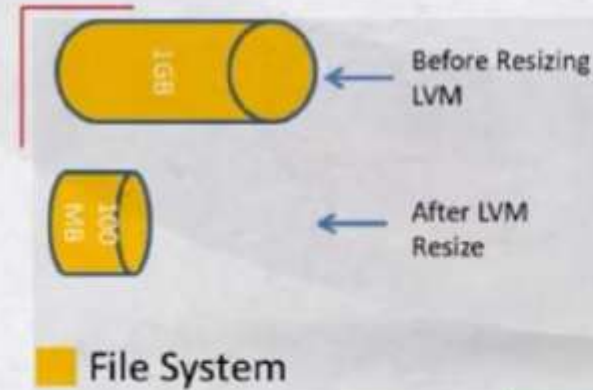
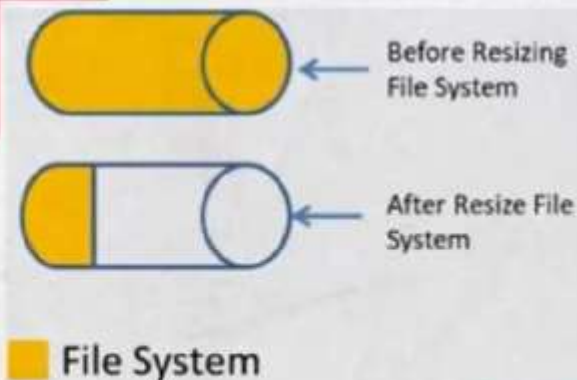
○ `lvextend -r -L +3G /dev/data-vg/data-lv`

Summary of LVM Commands

- Shrinking a volume group:
 - XFS doesn't support shrinking.

```
No modify flag set, skipping filesystem flush and exiting.  
fsadm: Xfs filesystem shrinking is unsupported.  
/usr/sbin/fsadm failed: 1  
Filesystem resize failed.
```

```
# umount /data  
# resize2fs /dev/data-vg/data-lv 100M  
# e2fsck -f /dev/data-vg/data-lv  
# lvreduce --size -900M /dev/data-vg/data-lv  
# lvreduce --size -r -900M /dev/data-vg/data-lv  
# vgreduce data-vg /dev/sde1 (removes sde1 from data-vg)  
# mount /dev/data-vg/data-lv /data
```



:Shrinking Logical Volumes (Downtime Needed)

- عملية تقليل حجم ال Logical Volume تتطلب بالضرورة توقف الخدمة (Down time is needed), مما يعني أنه يجب عليك عمل `umount` لل Logical Volume أولاً قبل تنفيذ أي خطوات.

قيود نظام الملفات XFS:

- إذا كان ال Logical Volume مهيناً بنظام الملفات XFS, فلن تتمكن نهائياً من إجراء تقليص أو تصغير له (XFS doesn't support shrinking), حيث يظهر خطأ يفيد بأن نظام ملفات XFS لا يدعم هذه العملية.

ترتيب خطوات التقليص (Shrink Order):

- في عملية التقليص، تقوم بالعكس تماماً مقارنة بالتوسيع؛ حيث تبدأ أولاً بتصغير ال File system ثم تليها خطوة تقليص ال Logical Volume نفسه.

التحقق من سلامة الملفات عبر أمر `e2fsck`:

- يتم تنفيذ أمر ال `e2fsck -f` للتحقق وفحص ما إذا كان ال File system تالفاً (corrupted) أم لا. هذا الأمر بالغ الأهمية لأن أي خطأ في حجم ال `resize` قد يؤدي إلى تلف نظام الملفات وفقدان البيانات.

- **توصية المحاضر:** يُفضل ويُرشح استخدام طريقة الخطوة الواحدة لتقليل الأخطاء لأنها تعتبر أكثر أماناً.

`umount /data` ○

تصغير نظام الملفات (Resize File System):

`resize2fs /dev/data-vg/data-lv 100M` ○

فحص سلامة الـ File System:

`e2fsck -f /dev/data-vg/data-lv` ○

تقليص حجم الـ Logical Volume (بالطريقة اليدوية):

`lvreduce --size -900M /dev/data-vg/data-lv` ○

تقليص الـ Logical Volume في خطوة واحدة (آمنة وموصى بها):

`lvreduce --size -r -900M /dev/data-vg/data-lv` ○

إزالة Physical Volume من الـ Volume Group (تقليص الـ VG):

○ قبل تصغير أو إزالة الـ PV من الـ VG باستخدام أمر الـ `vgreduce` , يجب التأكد تماماً أن الـ Physical Volume خالي تماماً من البيانات وأن المساحة المستخدمة عليه تساوي صفر (Allocated space = 0). إذا لم يكن كذلك، ستحتاج إلى نقل البيانات أولاً باستخدام أوامر النقل لضمان عدم ضياعها.

`vgreduce data-vg /dev/sde1` ○

إعادة التوصيل (Remount):

`mount /dev/data-vg/data-lv /data` ○

```

pvcreate /dev/nvme0n2 /dev/nvme0n3
vgcreate myvg01 /dev/nvme0n2 /dev/nvme0n3
vgs
lvcreate -n lv01-xfs -L 2G myvg01
lvs
lvcreate -n lv02-ext4 -L 2G myvg01
lvcreate -n lv03-swap -L 2G myvg01
lsblk

```

- عندما قمت بتجميع قرصين معا وهما `dev/nvme0n2/` و `dev/nvme0n3/` داخل الـ Volume Group المسمى `myvg01` ، فإن نظام الـ LVM يتعامل معهما كوعاء تخزيني موحد ومدمج (Storage Pool).
- عند إنشاء الـ Logical Volumes تباعاً، فإن الـ LVM يوزع المساحات بشكل تسلسلي خطي (Linear Allocation). هذا يعني أنه يبدأ باستهلاك المساحة من القرص الأول المتاح حتى يمتلئ، ثم ينتقل تلقائياً لتكملة أو وضع باقي الـ LVs على القرص التالي.
- سلوك النظام عند حدوث تقليص (Shrink) وظهور مساحة فارغة في القرص الأول: إذا افترضنا أن القرص الأول امتلئ وانتقل الـ LVM للقرص الثاني، ثم قمت أنت عمداً بعمل تصغير (Shrink) لـ Logical Volume معين على القرص الأول مما أدى إلى توفير مساحة فارغة جديدة (مثل 1 جيجا)، وحاولت بعدها إنشاء Logical Volume جديد بحجم صغير (مثل 500 ميجابايت)، فإن الـ LVM لن يعود أبداً للقرص الأول لاستغلال تلك المساحة الفارغة التي أفرغتها بالـ Shrink.
- السبب: يظل نظام الـ LVM الخطي ماشياً في اتجاه واحد للأمام، ولا يلتفت للوراء لملء الفراغات الناتجة عن التقليص أو الحذف تلقائياً، بل سيستمر في الاستهلاك من المساحات المتاحة في نهاية التسلسل الحالي (أي من القرص الثاني أو المساحات التالية) ما لم تقوم بإدارة التخزين أو نقل البيانات يدوياً.

- الشرح: يتم استخدام هذا الأمر لعرض الخصائص التفصيلية والدقيقة لكل ال Logical Volumes الموجودة في النظام (مثل الحجم، والحالة، وعدد ال Segments، والأهم من ذلك أنه يوضح مسار ال LV Path الخاص بكل فوليوم مثل `dev/myvg01/lv01-xfs/` و `dev/myvg01/lv02-ext4/`، وهو ما يتيح لك معرفة المسار الكامل الذي ستتعامل معه لاحقاً في عمليات ال Format وال Mount.

○ Initialize File Systems and Swap

○ `mkfs -t xfs /dev/myvg01/lv01-xfs`

○ `mkfs -t ext4 /dev/myvg01/lv02-ext4`

○ `mkswap /dev/myvg01/lv03-swap`

○ Mount and Bind Logical Volumes

○ `mkdir /lv01-xfs`

○ `mount /dev/myvg01/lv01-xfs /lv01-xfs`

○ `mkdir /lv02-ext4`

○ `mount /dev/myvg01/lv02-ext4 /lv02-ext4`

○ Swap Activation and Management

○ `free -m`

○ `swapon /dev/myvg01/lv03-swap`

○ `free -m`

`vgdisplay` ○

○ **الشرح:** يُستخدم هذا الأمر لعرض تفاصيل الـ VG بالكامل (مثل إجمالي المساحة VG Size، والمساحة الحرة Free PE / Size)، وهو ما تم تنفيذه للتأكد من وجود مساحة كافية قبل البدء في توسيع الـ Logical Volume.

خطوات عملية التوسيع وتحديث نظام الملفات:

`lvextend /dev/myvg01/lv01-xfs -L +2G` ○

`df -h` ○

`xfs_growfs /dev/myvg01/lv01-xfs` ○

○ **ملحوظة هامة على نتائج الأوامر:** عند تنفيذ أمر الـ `lvextend` وتوسيع الـ Logical Volume أولاً، لن يلاحظ نظام التشغيل أو المساحة المثبتة (`mounted`) أي تغير في الحجم عند فحص المساحات بالأمر `df -h` إلا بعد تنفيذ أمر تحديث نظام الملفات الخاص بنوع الـ File System (مثل `xfs_growfs` الذي تم تنفيذه في الخطوة التالية ليُجعل نظام التشغيل يرى المساحة الجديدة كاملة).

○ `vgdisplay` — نستخدم هذا الأمر للتأكد من حالة ومساحة الـ Volume Group، حيث نجد أن المساحة لم تعد تكفي لإجراء عمليات توسيع جديدة (Free PE / Size غير كافية)، مما يترتب عليه عدم إمكانية إكمال الـ Extend بالمساحة الحالية وحدوث نقص في الموارد المتاحة.

○ `lsblk` — لمعرفة الـ disks الموجودة في النظام والبحث للعثور على مساحة جديدة استخدمها في الـ `extend` (ضيف ديسك 5 جيجا).

`vgextend myvg01 /dev/nvme0n4` ○

`vgdisplay` ○

- الشرح: نلاحظ هنا أنه تم تنفيذ توسيع الـ Volume Group (`vgextend`) مباشرة دون الحاجة لعمل `pvcreeate` منفصلة في نفس اللحظة لأن النظام يتعامل معها أو يقوم بتهيئتها تلقائياً حسب السياق، وبعد تنفيذ الأمر زادت المساحة الإجمالية للـ VG بفضل القرص المضاف (`/dev/nvme0n4/`).

الصورة الرابعة (توسيع الـ Logical Volume الثاني):

○ الأمر المستخدم:

`lvextend /dev/myvg01/lv02-ext4 -L +2G` ○

`df -h` ○

- الشرح: تم تنفيذ عملية الـ `extend` للـ Logical Volume الثاني (`lv02-ext4`) بإضافة 2 جيجابايت جديدة، وعند فحص المساحات بالأمر `df -h` نلاحظ أن المساحة لم تتحدث في النظام بعد حتى يتم تحديث الـ file system الخاص به.

الصورة الخامسة (تحديث الـ File System لـ EXT4 ورؤية المساحة الجديدة):

○ الأوامر المستخدمة:

`resize2fs /dev/myvg01/lv02-ext4` ○

`df -h` ○

- الشرح: نظراً لأن هذا الـ Logical Volume يعمل بنظام ملفات EXT4 (وليس XFS)، تم استخدام أمر الـ `resize2fs` لتحديث الـ file system وجعله يتعرف على المساحة المضافة حديثاً، وبمراجعة الأمر `df -h` ظهرت المساحة وقد زادت بنجاح وأصبحت تظهر بالحجم الجديد.

`lvextend /dev/myvg01/lv03-swap -L +1G` ○

○ الشرح: يُستخدم هذا الأمر لتوسيع مساحة الـ Logical Volume الخاص بالـ Swap بإضافة 1 جيجابايت جديدة ليصبح حجمه الإجمالي 3 جيجابايت.

`free -m` ○

`mkswap /dev/myvg01/lv03-swap` ○

○ الشرح: حدث خطأ لأن الـ Swap مَفْعَل (`mounted` أو قيد الاستخدام حالياً)، وبالتالي لا يمكن عمل `mkswap` عليه وهو قيد التشغيل لحماية البيانات.

`swapoff /dev/myvg01/lv03-swap` ○

`free -m` ○

`mkswap /dev/myvg01/lv03-swap` ○

○ الشرح: بعد إيقافه، تم تنفيذ هذا الأمر بنجاح لتهيئة الـ Swap وتحديث المساحة الجديدة (3 جيجابايت) وكتابة الـ Swap signature الجديد.

`swapon /dev/myvg01/lv03-swap` ○

○ الشرح: يُستخدم لإعادة تشغيل وتفعيل مساحة الـ Swap بعد تعديل حجمها وتهيئتها.

`free -m` ○

○ الشرح: للتحقق النهائي والتأكد من أن مساحة الـ Swap أصبحت تعمل بكفاءة وظهرت بالحجم الجديد (4095 ميجابايت أي حوالي 4 جيجا).

`df -h` و `umount /lv01-xfs` و `lvdisplay`

- الشرح: في البداية تم استخدام `df -h` لعرض مساحات وأنظمة الملفات المتاحة، ثم تم إلغاء ربط (Unmount) ال Logical Volume الذي يعمل بنظام ملفات XFS (`lv01-xfs/`) لأنه لا يمكن تصغير مساحته وهو في حالة استخدام، وبعدها تم استخدام أمر `lvdisplay` لعرض تفاصيله ومعرفة اسمه ومساحته بدقة.

`lvreduce -r -L -2G /dev/myvg01/lv01-xfs`

- الشرح: ظهر خطأ عند محاولة تصغير مساحة ال Logical Volume الذي يعمل بنظام XFS ، لأن نظام ملفات XFS لا يدعم عملية التصغير (Shrink/Reduce) نهائياً، وظهرت رسالة تفيد بأن `File system reduce is required and not supported (xfs)` . ولحل هذه المشكلة (Workaround)، يتم أخذ نسخة احتياطية (Backup) من البيانات الموجودة عليه، ثم إعادة تهدأته (Format) بنظام ملفات آخر يدعم التصغير، ثم إجراء التصغير، ثم إعادته إلى XFS مرة أخرى ونقل البيانات إليه.

`df -h` و `mount /dev/myvg01/lv02-ext4 /lv02-ext4`

- الشرح: يتم إعادة ربط (Mount) ال Logical Volume الخاص بـ EXT4 والتحقق من المساحات وأنظمة الملفات باستخدام أمر `df -h` للتأكد من استقرار الأقراص وجاهزيتها للعمل.